

Github
 ACTUALIZADO HASTA LA SECCIÓN 11. Falta
 11.2.1, 11.2.2, 12, 13, etc.

Índice

0.1	OJO	1
1	Templates y Testing	1
1.1	Template y archivos	1
1.2	Logger	1
1.3	Stress testing (diff)	1
1.4	Stress testing (checker)	1
2	Ideas	1
3	Estructuras básicas	1
3.1	Min stack	1
3.2	Min queue	2
3.3	Heap actualizable	2
4	Teoría de números	2
4.1	Criba de Eratóstenes	2
4.2	Algoritmo extendido de Euclides	2
4.3	Solución de ecuaciones diofánticas lineales	2
4.4	Funciones multiplicativas	3
4.5	Factorización Pollard Rho	3
5	Combinatoria	3
5.1	Números de Catalán	3
5.2	Números de Narayana	4
5.3	Particiones Enteras	4
6	Cálculo	5
6.1	Transformada de Fourier	5
7	Formulazas	5
7.1	Lógica - Conjuntos - Bitwise	5
7.2	Combinatoria	5
7.3	Geometría	5
8	Métodos numéricos	6
8.1	Inversa de Matriz con Gauus-Jordan	6
8.2	Sistemas de ecuaciones lineales	6
8.3	Sistemas de ecuaciones modulo 2	6
9	Sparse table	6
10	Fenwick Tree	6
11	Segment Tree	7
11.1	Actualizaciones puntuales	7
11.2	Actualizaciones sobre rangos	7

11.3	Sparse segment tree	7
12	Sqrt decomposition	7
12.1	Algoritmo de MO	7
13	Grafos	7
13.1	Caminos mínimos	7
13.2	Árboles	8
13.3	Máximo flujo	9
13.4	SCC	11
13.5	2-Sat	11
14	Treap	11
15	Strings	12
15.1	KMP	12
15.2	Suffix Automata	12
15.3	Suffix array	12
15.4	Aho-Corasick	13
15.5	Suffix tree	13
16	Geometría	13
16.1	Convex hull	13
17	Utilidades	14
17.1	Subset sum optimization.	14
17.2	Bitsets de tamaño (casi) dinámico	14
17.3	Bitwise ops	14
17.4	Bitwise - builtin	14
17.5	Iterar	14
17.6	Gospers' Hack	14
17.7	Subset Sum con bitset	14
18	Máximo de funciones	14
18.1	Li-Chao Tree	14

► OJO

- Se usan macros (MAXN, LOGN, etc) con arreglos estáticos para más comodidad, pero puede causar RTE o MLE cuando los valores son grandes. Pensar en usar `vector<>` (STL) cuando sea conveniente.

TEMPLATES Y TESTING

► Template y archivos

template.cpp :

```
1 #include <bits/stdc++.h>
2 #include <ext/pb_ds/assoc_container.hpp>
3 #include <ext/pb_ds/tree_policy.hpp>
4 using namespace std;
5 using namespace __gnu_pbds;
6 typedef int64_t ll;
7 typedef pair<int, int> pii;
8 typedef pair<ll, ll> pll;
9 typedef vector<int> vi;
10 typedef vector<ll> vll;
11 typedef vector<vi> vvi;
12 typedef vector<vll> vvll;
13 typedef vector<pii> vpii;
14 typedef vector<pll> vpll;
15 typedef tree<int, null_type, less<int>, rb_tree_tag,
16     tree_order_statistics_node_update> ordered_set;
17 #define fi first
18 #define se second
19 #define pb push_back
20 #define all(x) (x).begin(), (x).end()
21 #define sz(x) (int)(x).size()
22 mt19937_64 gen(chrono::steady_clock::now().time_since_epoch().
23     count());
24 uniform_int_distribution<ll> distr(1, LLONG_MAX);
25 const int MOD = 1e9 + 7;
```

Despues de crear template.cpp, ejecutar en la terminal de carpeta

```
1 for f in {a..o}; do cp template.cpp $f.cpp;
```

► Logger

Para debuggear, llamamos al metodo $deb(x, y, z, \dots)$ con el numero de variables que ocupemos.

```
1 template<typename A, typename B> ostream& operator<<(ostream
2     & os, const pair<A, B> &p){return os << '(' << p.fi << ", "
3     << p.se << ')';}
4 template<typename C, typename T = typename
5     enable_if<!is_same<C, string>::value, typename
6     C::value_type>::type>
7 ostream& operator<<(ostream &os, const C &v){string sep;
8     for(const T &x : v) os << sep << x, sep = " "; return os;}
9 #define print(...) logger(__VA_ARGS__, __VA_ARGS__)
10 template<typename ...Args>
11 void logger(string vars, Args&&... values){
12     cout << "[Debug]\n\t" << vars << " = ";
13     string d = "[";
14     (... , (cout << d << values, d = "] ");
15     cout << "]\n";
```

► Stress testing (diff)

Para stress testing en problemas donde existe solo una respuesta, usamos el siguiente script y lo guardamos en

stress.sh. code.cpp será mi código a testear, *brute.cpp* el código bruto pero correcto y *gen.cpp* el generador de casos. Este ultimo recibirá como semilla el valor i .

```
1 set -e
2 g++ code.cpp -o code
3 g++ gen.cpp -o gen
4 g++ brute.cpp -o brute
5 for((i = 1; ; ++i)); do
6     ./gen $i > input_file
7     ./code < input_file > myAnswer
8     ./brute < input_file > correctAnswer
9     diff -Z myAnswer correctAnswer > /dev/null || break
10    echo "Passed test: " $i
11 done
12 echo "WA on the following test:"
13 cat input_file
14 echo "Your answer is:"
15 cat myAnswer
16 echo "Correct answer is:"
17 cat correctAnswer
```

Para extraer la semilla en gen.cpp leemos desde argv

```
1 int rnd(int a, int b){
2     return a + rand() % (b - a + 1);
3 }
4 int main(int argc, char* argv[]){
5     int seed = atoi(argv[1]);
6     srand(seed);
7     int n = rnd(1, 5);
8 }
```

► Stress testing (checker)

Usaremos el script similar al anterior *stress.sh*. Aquí el *brute.cpp* es opcional.

```
1 set -e
2 g++ code.cpp -o code
3 g++ gen.cpp -o gen
4 g++ brute.cpp -o brute
5 g++ checker.cpp -o checker
6 for((i = 1; ; ++i)); do
7     ./gen $i > input_file
8     ./code < input_file > myAnswer
9     ./brute < input_file > correctAnswer
10    ./checker > checker_log
11    echo "Passed test: " $i
12 done
```

Se ejecutara mientras el return value de checker sea 0. Ejemplo de la implementación de un checker

```
1 bool validAns(ifstream &fin, int n, int* arr){
2     fin>>n;
3     //aquí validariamos nuestra respuesta, fin sera el input
4     stream que le pase
5     if(algunacosa) return true
6     else return false
7 }
8 int main(int argc, char * argv[]){
9     //input stream del caso
10    ifstream fin("input_file", ifstream::in);
11    //input stream de mi respuesta
12    ifstream ans("myAnswer", ifstream::in);
13    //input stream de respuesta bruta
14    ifstream cor("correctAnswer", ifstream::in);
15    //aquí fin puede procesar lo que me interese del input
16    original
17    fin>>n;
```

```
17 int arr[n];
18 for(int x = 0; x < n; x++) cin>>arr[x];
19 if(!validAns(ans, n, arr)) {
20     cout<<"WA x algun motivo, esto saldrá en checker_log\n";
21     return -1;
22 }
23 //si todo salio bien, retorno 0
24 return 0;
25 }
```

IDEAS

Ideas de Catalán. Ideas que se usan en las demostraciones de que algo se cuenta con Catalán y sirven para conteos similares:

- Partir en dos conjuntos y distribuir.* Usar la misma idea de la recurrencia de los números de Catalán: $(0, n-1), (1, n-2), \dots, (k, n-1-k)$.
- Reflejar los caminos malos.* Si queremos contar caminos monótonos que $(0, 0)$ y (n, m) , por debajo de una diagonal paralela a la que une dichas esquinas, en una cuadrícula de $n \times m$, contemos todos los caminos $\binom{n+m}{n}$ y restemos los caminos que pasan sobre la diagonal. Digamos que la diagonal es $y = x + k$, entonces los caminos malos pasan al menos una vez por $y = x + k + 1$: toma el primer punto $(x', x' + k + 1)$ sobre el que un camino pasa por $y = x + k + 1$ y refléjalo, nota que todos empiezan en $(x', x' + k + 1) - (x' + k + 1, x') = (-k - 1, k + 1)$. Entonces los caminos malos son biyectivos a los caminos que van de $(-k - 1, k + 1)$ a (n, m) .

Optimización. Ideas para optimizar funciones.

- Fijar coordenadas secuencialmente.* Hay funciones en R^n que se pueden optimizar fijando la primera coordenada y resolviendo para R^{n-1} y así sucesivamente.

ESTRUCTURAS BÁSICAS

► Min stack

```
1 template<typename T> struct min_stack{
2     stack<pair<T, T>> st;
3     min_stack(){}
4     min_stack(const T &MAXVAL){init(MAXVAL);}
5     void init(const T &MAXVAL){
6         st.push(make_pair(MAXVAL, MAXVAL));}
7     void push(const T &v){st.push(make_pair(v, min(v,
8         v - st.top().se));)}
9     T top(){return st.top().fi;}
10    void pop(){if(sz(st) > 1)st.pop();}
11    T minV(){return st.top().se;}
12    int size(){return sz(st) - 1;}
13    bool empty(){return size() == 0;}
```

► Min queue

```

1 template<typename T> struct min_queue{
2     min_queue(const T &MAXVAL){
3         p_in.init(MAXVAL); p_out.init(MAXVAL);}
4     void push(const T &v){p_in.push(v);}
5     T front(){transfer(); return p_out.top();}
6     void pop(){transfer(); p_out.pop();}
7     int size(){return sz(p_in)+sz(p_out);}
8     T minV() {
9         return min(p_in.minV(), p_out.minV());}
10    bool empty(){ return size() == 0;}
11    void transfer(){
12        if(sz(p_out)) return;
13        while(sz(p_in)){
14            p_out.push(p_in.top());
15            p_in.pop();
16        }
17    } min_stack<T> p_in, p_out;
18 };

```

► Heap actualizable

```

1 template<class TPriority, class TKey> class UpdatableHeap{
2 public:
3     UpdatableHeap(){
4         TPriority a;
5         TKey b;
6         nodes.clear();
7         nodes.pb( make_pair(a, b) );
8     }
9     pair<TPriority, TKey> top() {return nodes[1];}
10    void pop(){
11        if(sz(nodes) == 1) return;
12        TKey k = nodes[1].se;
13        swap_nodes(1, sz(nodes) - 1);
14        nodes.pop_back();
15        position.erase(k);
16        heapify(1);
17    }
18    void insert_or_update(const TPriority &p, const TKey &k){
19        int pos;
20        if(is_inserted(k)){
21            pos = position[k];
22            nodes[pos].fi += p;
23        } else {
24            position[k] = pos = sz(nodes);
25            nodes.pb( make_pair(p, k) );
26        }
27        heapify(pos);
28    }
29    bool is_inserted(const TKey &k) {
30        return position.count(k);
31    }
32    int get_size() {
33        return sz(nodes) - 1;
34    }
35    void erase(const TKey &k){
36        if(!is_inserted(k)) return;
37        int pos = position[k];
38        swap_nodes(pos, sz(nodes) - 1);
39        nodes.pop_back();
40        position.erase(k);
41        heapify(pos);
42    }
43 private:
44     vector<pair<TPriority, TKey>> nodes;
45     map<TKey, int> position;
46     void heapify(int pos){
47         if(pos >= sz(nodes)) return;
48         while(1 < pos && nodes[pos / 2] <= nodes[pos]){
49             swap_nodes(pos / 2, pos);
50             pos /= 2;
51         }
52         int l = pos * 2, r = pos * 2 + 1, maxi = pos;
53         if(l < sz(nodes) && nodes[l] > nodes[maxi]) maxi = l;
54         if(r < sz(nodes) && nodes[r] > nodes[maxi]) maxi = r;
55         if(maxi != pos){
56             swap_nodes(pos, maxi);

```

```

57         heapify(maxi);
58     }
59 }
60 void swap_nodes(int a, int b){
61     position[ nodes[a].se ] = b;
62     position[ nodes[b].se ] = a;
63     swap(nodes[a], nodes[b]);
64 }
65 };

```

TEORÍA DE NÚMEROS

► Criba de Eratóstenes

Criba Complejidad: Tiempo $O(n \log \log n)$ - Memoria extra $O(n)$. Calcula los primos menores o iguales a n .

```

1 void criba(int n, vi &primos){
2     primos.clear();
3     if(n < 2) return;
4     vector<bool> no_primo(n + 1);
5     no_primo[0] = no_primo[1] = true;
6     for(ll i = 3; i * i <= n; i += 2){
7         if(no_primo[i]) continue;
8         for(ll j = i * i; j <= n; j += 2 * i)
9             no_primo[j] = true;
10    }
11    primos.pb(2);
12    for(int i = 3; i <= n; i += 2)
13        if(!no_primo[i]) primos.pb(i);
14 }

```

Criba sobre un rango. Complejidad: Tiempo $O(\sqrt{b} \log \log \sqrt{b} + (b - a) \log \log (b - a))$ - Memoria extra $O(\sqrt{b} + b - a)$. Calcula los primos en el intervalo $[a, b]$.

```

1 void criba_rango(ll a, ll b, vector<ll> &primos){
2     a = max(a, 0ll);
3     b = max(b, 0ll);
4     ll tam = b - a + 1;
5     vi primos_raiz;
6     criba(sqrt(b) + 1, primos_raiz);
7     bool no_primo[tam] = {};
8     primos.clear();
9     for(ll p : primos_raiz){
10        ll ini = p * max(p, (a + p - 1) / p);
11        for(ll m = ini; m <= b; m += p)
12            no_primo[m - a] = true;
13    }
14    for(ll i = 0; i < tam; ++i)
15        if(!(no_primo[i] || i + a < 2))
16            primos.pb(i + a);
17 }

```

Criba segmentada. Complejidad: Tiempo $O(\sqrt{n} \log \log \sqrt{n} + n \log \log n)$ - Memoria extra $O(\sqrt{n} + S)$. Cuenta la cantidad de primos menores o iguales a n .

```

1 int cuenta_primos(int n){
2     if(n < 2) return 0;
3     const int S = sqrt(n);
4     vi primos_raiz;
5     criba(sqrt(n) + 1, primos_raiz);
6     int ans = 0;
7     bool no_primo[S + 1] = {};
8     for(int ini = 0; ini <= n; ini += S){
9         memset(no_primo, 0, S + 1);
10        for(int p : primos_raiz){
11            int m = p * max(p, (ini + p - 1) / p) - ini;

```

```

12        for(; m <= S; m += p) no_primo[m]=1;
13    }
14    for(int i=0; i<S && i + ini <= n; ++i)
15        if(!no_primo[i] && 1 < i + ini) ans++;
16    } return ans;
17 }

```

Criba lineal. Complejidad: Tiempo $O(n)$ - Memoria extra $O(n)$. Calcula los primos menores o iguales a n y el menor primo que divide a cada entero en $[2, n]$. ADVERTENCIA: es $O(n)$ pero tiene una constante grande.

```

1 void criba_lineal(int n, vi &primos){
2     if(n < 2) return;
3     vi lp(n + 1);
4     for(ll i = 2; i <= n; ++i){
5         if(!lp[i]) primos.pb(lp[i] = i);
6         for(int j = 0; i * primos[j] <= n; ++j){
7             lp[i * primos[j]] = primos[j];
8             if(primos[j] == lp[i]) break;
9         }
10    }
11 }

```

► Algoritmo extendido de Euclides

Complejidad: Tiempo $O(\log(\max(a, b)))$ - Memoria extra $O(1)$. Encuentra una solución a la ecuación $ax + by = \gcd(a, b)$.

```

1 int gcd_ext(int a, int b, int &x, int &y){
2     if(!b){ x = 1; y = 0; return a; }
3     int x1, y1, g = gcd_ext(b, a % b, x1, y1);
4     x = y1;
5     y = x1 - y1 * (a / b);
6     return g;
7 }

```

► Solución de ecuaciones diofánticas lineales

Complejidad: Tiempo $O(\log(\max(a, b)))$ - Memoria extra $O(1)$. Encuentra una solución a la ecuación $ax + by = c$ o determina si no existe solución.

```

1 bool encuentra_solucion(int a, int b, int c, int &x, int &y,
2     int &g){
3     g = gcd_ext(abs(a), abs(b), x, y);
4     if(c % g) return false;
5     x *= c / g;
6     y *= c / g;
7     if(a < 0) x = -x;
8     if(b < 0) y = -y;
9     return true;

```

Cambia a la siguiente (anterior) solución $|cnt|$ veces. $g := \gcd(a, b)$.

```

1 void cambia_solucion(int &x, int &y, int a, int b, int cnt,
2     int &g = 1) {
3     x += cnt * b / g;
4     y -= cnt * a / g;

```

Cuenta la cantidad de soluciones x, y con $x \in [minx, maxx]$ y $y \in [miny, maxy]$.

```

1 int cuenta_soluciones(int a, int b, int c, int minx, int maxx,
  ↪ int miny, int maxy) {
2     int x, y, g;
3     if(!encuentra_solucion(a, b, c, x, y, g)) return 0;
4     // ax + by = c ssi (a/g)x + (b/g)y = c/g
5     // Dividimos entre g para simplificar y no dividir a cada
  ↪ rato
6     a /= g;
7     b /= g;
8     // Signos de a, b nos sirven para pasar a la
9     // siguiente (anterior) solucion
10    int sign_a = a > 0 ? +1 : -1;
11    int sign_b = b > 0 ? +1 : -1;
12    // pasa a la minima solucion tal que minx <= x
13    cambia_solucion(x, y, a, b, (minx - x) / b);
14    // si x < minx, pasa a la siguiente para que minx <= x
15    if(x < minx) cambia_solucion(x, y, a, b, sign_b);
16    if(x > maxx) return 0; // si x > maxx, entonces no hay x
  ↪ solucion tal que x in [minx, maxx]
17    int lx1 = x;
18    // pasa a la maxima solucion tal que x <= maxx
19    cambia_solucion(x, y, a, b, (maxx - x) / b);
20    if(x > maxx) cambia_solucion(x, y, a, b, -sign_b); // si x
  ↪ > maxx, pasa a la solucion anterior
21    int rx1 = x;
22    // hace todo lo anterior pero con y
23    cambia_solucion(x, y, a, b, -(miny - y) / a);
24    if(y < miny) cambia_solucion(x, y, a, b, -sign_a);
25    if(y > maxy) return 0;
26    int lx2 = x;
27    cambia_solucion(x, y, a, b, -(maxy - y) / a);
28    if(y > maxy) cambia_solucion(x, y, a, b, sign_a);
29    int rx2 = x;
30    // como al encontrar las x tomando y como criterio no nos
  ↪ asegura
31    // que esten ordenadas, entonces las ordenamos
32    if(lx2 > rx2) swap(lx2, rx2);
33    // obtenemos la interseccion de los intervalos
34    int lx = max(lx1, lx2);
35    int rx = min(rx1, rx2);
36    if(lx > rx) return 0; // no existen soluciones,
  ↪ interseccion vacia
37    // las soluciones (por x) van de b en b (b/g en b/g pero
  ↪ dividimos al principio)
38    return (rx - lx) / abs(b) + 1;
39 }

```

► Funciones multiplicativas

Función Phi de Euler. Complejidad: Tiempo $O(d)$ - Memoria extra $O(n)$. d es la cantidad de factores primos de n . Cuenta la cantidad de coprimos con n menores a n .

```

1 int phi(int n){
2     if(n <= 1) return 1;
3     if(!dp[n]){
4         int pot = 1, p = lp[n], n0 = n;
5         while(n0 % p == 0){ pot *= p; n0 /= p; }
6         dp[n] = (pot / p) * (p - 1) * phi(n0);
7     } return dp[n];
8 }

```

Función σ_0 . Complejidad: Tiempo $O(d)$ - Memoria extra $O(n)$. d es la cantidad de factores primos de n . Cuenta la cantidad de divisores de n .

```

1 ll sigma0(int n){
2     if(n <= 1) return 1;
3     if(!dp[n]){
4         ll exp = 0, p = lp[n], n0 = n;
5         while(n0 % p == 0){ exp++; n0 /= p; }
6         dp[n] = (exp + 1) * sigma0(n0);
7     } return dp[n];
8 }

```

Función σ_1 . Complejidad: Tiempo $O(d)$ - Memoria extra $O(n)$. d es la cantidad de factores primos de n . Calcula la suma de los divisores de n .

```

1 ll sigma1(int n){
2     if(n <= 1) return 1;
3     if(!dp[n]){
4         ll pot = 1, p = lp[n], n0 = n;
5         while(n0 % p == 0){ pot *= p; n0 /= p; }
6         dp[n] = (pot*p - 1) / (p - 1) * sigma1(n0);
7     } return dp[n];
8 }

```

Función de Moebius. Complejidad: Tiempo $O(d)$ - Memoria extra $O(n)$. d es la cantidad de factores primos de n . Devuelve 0 si n no es divisible por algún cuadrado. Devuelve 1 o -1 si n es divisible por al menos un cuadrado. Devuelve 1 si n tiene una cantidad par de factores primos. Devuelve -1 si n tiene una cantidad impar de factores primos.

```

1 int moebius(int n){
2     if(n <= 1) return 1;
3     if(dp[n] == -7){
4         int exp = 0, p = lp[n], n0 = n;
5         while(n0 % p == 0){ exp++; n0 /= p; }
6         dp[n] = (exp > 1 ? 0 : -1 * moebius(n0));
7     } return dp[n];
8 }

```

► Factorización Pollard Rho

Complejidad: $O(\sqrt[n]{n})$. Inicializar con PollardRho::init() y para factorizar un número PollardRho::factorize(n).

```

1 //COMPIADO Y PEGADOPORCUESTIONES DE TIEMPO
2 namespace PollardRho {
3     mt19937 rnd(chrono::steady_clock::now()
4         .time_since_epoch().count());
5     const int P = 1e6 + 9;
6     ll seq[P];
7     int primes[P], spf[P];
8     inline ll add_mod(ll x, ll y, ll m){
9         return (x + y) < m ? x + y - m; }
10    inline ll mul_mod(ll x, ll y, ll m) {
11        ll res = __int128(x) * y % m;
12        return res;
13        // ll res = x * y - (ll)((long double)x * y / m + 0.5) *
  ↪ m;
14        // return res < 0 ? res + m : res;
15    }
16    inline ll pow_mod(ll x, ll n, ll m) {
17        ll res = 1 % m;
18        for (; n >= 1) {
19            if (n & 1) res = mul_mod(res, x, m);
20            x = mul_mod(x, x, m);
21        }
22        return res;
23    }
24    // 0(it * (logn)^3), it = number of rounds performed
25    inline bool miller_rabin(ll n) {
26        if (n <= 2 || (n & 1 ^ 1)) return (n==2);
27        if (n < P) return spf[n] == n;
28        ll c, d, s = 0, r = n - 1;
29        for (; !(r & 1); r >>= 1, s++) {}
30        // each iteration is a round
31        for(int i=0; primes[i]<n && primes[i]<32; i++){
32            c = pow_mod(primes[i], r, n);
33            for(int j = 0; j < s; j++){

```

```

34                d = mul_mod(c, c, n);
35                if(d==1&&c!=1&&c!=n-1) return 0;
36                c = d;
37                } if (c!=1) return 0;
38            } return 1;
39        }
40    void init() {
41        int cnt = 0;
42        for(int i = 2; i < P; i++){
43            if(!spf[i]) primes[cnt++] = spf[i]=i;
44            for(int j=0; k=(i*primes[j])<P; j++){
45                spf[k] = primes[j];
46                if (spf[i] == spf[k]) break;
47            }
48        }
49        // returns 0(n^(1/4))
50        ll pollard_rho(ll n) {
51            while(1){
52                ll x=rnd()%n,y=x,c=rnd()%n,u=1,v,t=0;
53                ll *px = seq, *py = seq;
54                while(1){
55                    *py++ = y = add_mod(mul_mod(y, y, n), c, n);
56                    *py++ = y = add_mod(mul_mod(y, y, n), c, n);
57                    if((x = *px++) == y) break;
58                    v = u;
59                    u = mul_mod(u, abs(y - x), n);
60                    if(!u) return __gcd(v, n);
61                    if(++t == 32){
62                        t = 0;
63                        if((u = __gcd(u, n)) > 1 && u < n)
64                            return u;
65                    }
66                }
67                if(t && (u = __gcd(u, n)) > 1 && u < n)
68                    return u;
69            }
70        }
71    }
72    vector<ll> factorize(ll n) {
73        if(n == 1) return vector<ll>();
74        if(miller_rabin(n)) return vector<ll>{n};
75        vector<ll> v, w;
76        while(n > 1 && n < P){
77            v.pb(spf[n]); n /= spf[n];
78        }
79        if(n >= P){
80            ll x = pollard_rho(n);
81            v = factorize(x);
82            w = factorize(n / x);
83            v.insert(v.end(), all(w));
84        } return v;
85    }
86 }

```

COMBINATORIA

► Números de Catalán

Se puede calcular con $C_0 = C_1 = 1$,

$$C_n = \sum_{k=0}^{n-1} C_k C_{n-1-k} = \frac{1}{n+1} \binom{2n}{n} = \frac{4n+2}{n+2} C_{n-1}, \quad n \geq 2.$$

El n -ésimo número de Catalán C_n cuenta

- La cantidad de secuencias de paréntesis balanceadas de longitud $2n$.
- La cantidad de maneras distintas de agrupar $n + 1$ factores con paréntesis.

- La cantidad de triangulaciones de un polígono convexo de $n + 2$ lados.
- La cantidad de maneras de unir $2n$ puntos en una circunferencia con cuerdas sin que ningún par se corte.
- La cantidad de árboles binarios completos con n nodos *internos* no isomorfos. Los nodos *internos* son aquellos con dos hijos.
- La cantidad de árboles binarios enraizados completos no isomorfos con $n + 1$ hojas.
- La cantidad de árboles enraizados planos no isomorfos con $n + 1$ nodos.
- La cantidad de árboles binarios no isomorfos con exactamente n nodos.
- La cantidad de caminos monótonos en un tablero de $n \times n$ que van de $(0,0)$ a (n,n) sin que cruce la diagonal que une $(0,0)$ con (n,n) .
- La cantidad de permutaciones de tamaño n que son ordenables con una pila (mientras $\text{top}() \leq x$, $\text{pop}()$. Luego $\text{push}(x)$. Al final haz $\text{pop}()$ de los elementos restantes de la pila). Equivalentemente la cantidad de permutaciones que no contienen el patrón 231: no existen índices $i < j < k$ tales que $a_k < a_i < a_j$.
- La cantidad de permutaciones de tamaño n que no contienen el patrón 123: no existen índices $i < j < k$ tales que $a_i < a_j < a_k$.
- La cantidad de particiones no cruzadas de un conjunto de tamaño n .
- La cantidad de maneras de cubrir una escalera con n escalones, con la altura del i -ésimo escalón siendo i , mediante n rectángulos.
- La cantidad de maneras de unir n cuadros de 1×1 tales que cada cuadro tenga a otro cuadro adyacente a sus lados y, cada columna de cuadros tenga una altura absoluta mayor o igual a la altura absoluta a la columna previa. Cada uno de estos polígonos tiene un perímetro de $2n + 2$.

Triángulo de Catalán. Sean $n, k \in \mathbb{Z}_{\geq 0}$, definamos

$$C_{n,k} = \begin{cases} 0, & n < k \text{ o } n, k < 0, \\ 1, & n = k = 0, \\ C_{n,k-1} + C_{n-1,k}, & k \leq n. \end{cases}$$

Entonces $C_{n,n} = C_n = \sum_{k=1}^{n-1} C_{n-1,k}$. El número $C_{n,k}$ se puede interpretar como la cantidad de caminos desde (n,k) del Triángulo de Catalán hasta $(0,0)$. Por lo que tiene sentido que $C_{n,n} = C_n$, dado que $C_{n,n}$ es la cantidad de caminos monótonos desde (n,n) a $(0,0)$ en un tablero de $n \times n$ que no cruzan la diagonal que une $(0,0)$ y (n,n) .

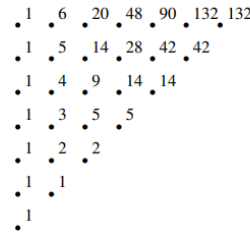


Figura 5.1: Triángulo de Catalán hasta $n = 6$.

► Números de Narayana

Sean $n, k \in \mathbb{Z}^+$ con $k \leq n$, los números de Narayana se definen como

$$N(n, k) = \frac{1}{n} \binom{n}{k} \binom{n}{k-1}.$$

Se cumple que

$$N(n, k) = N(n, n - k + 1),$$

$$\sum_{k=1}^n N(n, k) = C_n,$$

donde C_n es el n -ésimo número de Catalán. El número $N(n, k)$ cuenta

- La cantidad de secuencias de paréntesis balanceadas con $2n$ paréntesis y k distintos anidamientos, es decir, k ocurrencias de la subcadena $()$.
- La cantidad de caminos distintos desde $(0,0)$ a $(2n,0)$ dando pasos hacia arriba o abajo y siempre avanzando una unidad en cada paso (diagonales), de manera que haya exactamente k picos. Equivalentemente

a la cantidad de maneras de ordenar n 1's y n (-1) 's en una secuencia a_i tales que si S_i es la suma parcial de a_i , entonces $\{S_i\}$ tiene exactamente k máximos locales.

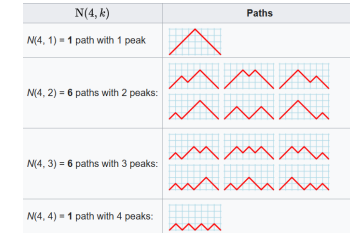


Figura 5.2: Caminos de $(0,0)$ a $(8,0)$ que cuenta $N(4,k)$.

- La cantidad de árboles enraizados con $n + 1$ nodos y k hojas.
- La cantidad de particiones no cruzadas de un conjunto de tamaño n en exactamente k subconjuntos.

► Particiones Enteras

Genera todas las particiones de un número n como suma de enteros positivos en orden no creciente. Complejidad: Tiempo $O\left(\frac{13^{\sqrt{n}}}{n}\right)$ – Memoria extra $O(n)$.

```
1 void generar(int n, int maximo, vi &actual){
2   if(n == 0){// Aquí trabajar con actual
3     return; }
4   for(int i = min(n, maximo); i >= 1; --i){
5     actual.pb(i);
6     generar(n - i, i, actual);
7     actual.pop_back();
8   }
9 }
```

La función de partición $p(n)$, que cuenta el número de formas distintas de escribir n como suma de enteros positivos sin importar el orden,

n	1	2	3	4	5	6	7	8
$p(n)$	1	2	3	5	7	11	15	22
n	9	10	11	12	13	14	15	16
$p(n)$	30	42	56	77	101	135	176	231

Tabla 1: Particiones enteras

Tiene la siguiente aproximación asintótica para valores grandes de n , conocida como la fórmula de Hardy-Ramanujan:

$$p(n) \sim \frac{1}{4n\sqrt{3}} \exp\left(\pi\sqrt{\frac{2n}{3}}\right)$$

CÁLCULO

► Transformada de Fourier

FFT. Complejidad: Tiempo $O(n \log n)$ - Memoria extra $O(n \log n)$, donde n es el grado del polinomio P .

```
1 using comp = complex<double>;
2 const double PI = acos(-1);
3 vector<comp> FFT(vector<comp> &P, bool inversa){
4     int n = sz(P);
5     if(n == 1) return P;
6     vector<comp> Pe, Po;
7     for(int i = 0; i < n; ++i)
8         if(i % 2) Po.pb(P[i]);
9         else Pe.pb(P[i]);
10    vector<comp> eval_Pe = FFT(Pe, inversa);
11    vector<comp> eval_Po = FFT(Po, inversa);
12    vector<comp> eval(n);
13    double angulo = 2*PI / n * (inversa? -1 : 1);
14    comp w(1), w_n(cos(angulo), sin(angulo));
15    for(int i = 0; i < n / 2; ++i){
16        eval[i] = eval_Pe[i] + w * eval_Po[i];
17        eval[i+n/2] = eval_Pe[i] - w*eval_Po[i];
18        if(inversa){eval[i]/=2; eval[i+n/2]/=2;}
19        w *= w_n;
20    } return eval;
21 }
```

Multiplicar polinomios. Complejidad: Tiempo $O(n \log n)$ - Memoria extra $O(n \log n)$, donde n es el grado máximo del polinomio A o B .

```
1 vi multiplicar(vi A, vi B){
2     vector<comp> cA(all(A)), cB(all(B));
3     int n = 1;
4     while(n < sz(A) + sz(B)) n *= 2;
5     cA.resize(n);
6     cB.resize(n);
7     vector<comp> val_A = FFT(cA, false);
8     vector<comp> val_B = FFT(cB, false);
9     for(int i=0; i<n; ++i) val_A[i] *= val_B[i];
10    val_A = FFT(val_A, true);
11    vi res(n);
12    for(int i = 0; i < n; ++i)
13        res[i] = round(val_A[i].real());
14    int carry = 0;
15    for(int i = 0; i < n; i++){ res[i] += carry;
16        carry = res[i] / 10; res[i] %= 10;
17    } return res;
18 }
```

FORMULAZAS

► Lógica - Conjuntos - Bitwise

$\oplus$ es el xor (o diferencia simétrica de conjuntos).

1. $a|b = a \oplus b + a\&b$

2. $a \oplus (a\&b) = (a|b) \oplus b$
3. $b \oplus (a\&b) = (a|b) \oplus a$
4. $(a\&b) \oplus (a|b) = a \oplus b$
5. $a + b = a|b + a\&b = a \oplus b + 2(a\&b)$
6. $a - b = (a \oplus (a\&b)) - ((a|b) \oplus a) = ((a|b) \oplus b) - ((a|b) \oplus a)$
7. $a - b = (a \oplus (a\&b)) - (b \oplus (a\&b)) = ((a|b) \oplus b) - (b \oplus (a\&b))$

Si p, q son predicados, entonces:

1. $(p \vee q) \iff \neg(\neg p \wedge \neg q).$
2. $(p \implies q) \iff (\neg p \vee q).$
3. $(p \oplus q) \iff \neg(p \iff q).$

► Combinatoria

Inclusión-Exclusión. Sean $A_1, \dots, A_n$ conjuntos, entonces

$$\left| \bigcup_{i=1}^n A_i \right| = \sum_{\emptyset \neq J \subseteq [n]} (-1)^{|J|-1} \left| \bigcap_{j \in J} A_j \right|$$

Cantidad de elementos en exactamente r conjuntos. Sean $A_1, \dots, A_n$ conjuntos y $0 \leq r \leq n$, la cantidad de elementos en exactamente r conjuntos A_i es

$$\sum_{m=r}^n (-1)^{m-r} \binom{m}{r} \sum_{\substack{J \subseteq [n] \\ |J|=m}} \left| \bigcap_{j \in J} A_j \right|$$

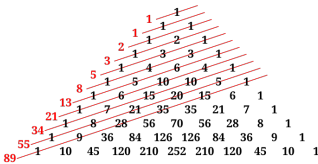
Teorema del binomio generalizado. Para $x \in \mathbb{R}$ y $a, b \in \mathbb{C}$ se cumple

$$(a + b)^x = \sum_{k=0}^{\infty} \binom{x}{k} a^x b^{x-k},$$
$$\binom{x}{k} = \frac{1}{k!} \prod_{j=0}^{k-1} (x - j) = \frac{x(x-1)(x-2) \cdots (x-k+1)}{k!}.$$

ES FIBONACCI (En un icpc, el de la Cangurera lo usó). En el *Triángulo de Pascal*, los elementos en una “diagonal” satisfacen

$$F_{n+1} = \sum_{k=0}^{\lfloor n/2 \rfloor} \binom{n-k}{k},$$

donde F_{n+1} es el $(n + 1)$ -ésimo número de Fibonacci.



Palo de Hockey. Para n y r ($n \geq r$), se cumple

$$\sum_{k=r}^n \binom{k}{r} = \binom{n+1}{r+1}.$$

Derangements. ¿Cuántos trastornos mentales distintos de tamaño n hay? (Según Google Translator). Considera los conjuntos $\{A_k\}_{k=1}^{k=n}$ donde A_k es la cantidad de permutaciones con el punto fijo $p_k = k$. La cantidad de permutaciones con al menos un punto fijo es

$$P := \left| \bigcup_{i=1}^n A_i \right| = \sum_{\emptyset \neq J \subseteq [n]} (-1)^{|J|-1} \left| \bigcap_{j \in J} A_j \right|$$
$$= \sum_{\emptyset \neq J \subseteq [n]} (-1)^{|J|-1} (n - |J|)! = \sum_{i=1}^n (-1)^{i-1} \binom{n}{i} (n - i)!,$$

entonces la cantidad de desarreglos es $n! - P$. Si d_n es la cantidad de desarreglos de tamaño n se cumple $d_n = (n - 1)(d_{n-1} + d_{n-2})$ con $d_1 = 0$ y $d_0 = d_2 = 1$. También se cumple $d_n = n!d_{n-1} + (-1)^n$. Sea $0 \leq k \leq n$ y $d_{n,k}$ la cantidad de desarreglos de tamaño n con exactamente k puntos fijos, entonces $d_{n,k} = \binom{n}{k} d_{n-k}$.

► Geometría

Teorema de los círculos de Descartes. Si cuatro círculos son mutuamente tangentes de curvatura $k_i = 1/r_i$ el teorema dice:

$$(k_1 + k_2 + k_3 + k_4)^2 = 2(k_1^2 + k_2^2 + k_3^2 + k_4^2)$$
$$k_4 = k_1 + k_2 + k_3 \pm 2\sqrt{k_1k_2 + k_2k_3 + k_3k_1}$$

Formula de Heron. Dado un triángulo de lados a, b, c , escribimos su semiperimetro $s = \frac{a+b+c}{2}$, inradio r , y cui-runradio R .

$$Area = \sqrt{s(s-a)(s-b)(s-c)}$$
$$2R = \frac{abc}{\sqrt{s(s-a)(s-b)(s-c)}}$$

$$r = \sqrt{\frac{(s-a)(s-b)(s-c)}{s}}$$

Fórmula de Brahmagupta. Para el Area se vale algo similar en cuadrilateros ciclicos

$$Area = \sqrt{(s-a)(s-b)(s-c)(s-d)}$$

Para no cíclicos, θ = promedio de dos ángulos opuestos, p y q diagonales

$$Area = \sqrt{(s-a)(s-b)(s-c)(s-d) - abcd \cos^2 \theta} = \sqrt{K},$$

donde $K = (s-a)(s-b)(s-c)(s-d) - \frac{1}{4}(ac+bd+pq)(ac+bd-pq)$.

Multidimensional (Como Chang). Para más dimensiones, si A es la matriz con filas estos vectores

$$Volumen_n = \frac{1}{n!} \sqrt{\det(AA^T)}$$

MÉTODOS NUMÉRICOS

► Inversa de Matriz con Gauus-Jordan

Complejidad: Tiempo $O(n^3)$ - Memoria $O(n^2)$

```
1 #define eps 0.0001
2 vector<vector<double>> gauss(vector<vector<double>> m){
3     int tam = sz(m);
4     vector<vector<double>> id(tam, vector<double>(tam));
5     for(int i=0; i<tam; i++) id[i][i] = 1;
6     for(int i=0; i<tam; i++){
7         // buscar un no 0 para poner en (i,i)
8         if(abs(m[i][i]) < eps){
9             int j=i+1;
10            while(abs(m[j][i]) > eps && j<tam){
11                swap(m[j], m[i]);
12                swap(id[j], id[i]); j++;
13            }
14            if(j == tam){/*...*/}no invertible
15        }
16        /// poner un uno en (i,i) /// modificar la fila i
17        double tmp = m[i][i];
18        for(int k=0; k<tam; k++)
19            id[i][k] /= tmp, m[i][k] /= tmp;
20        /// poner columna i en todo 0, excepto casilla (i,i)
21        for(int j=0; j<tam; j++){
22            if(j==i) continue;
23            //a la fila j le restamos la fila i
24            tmp = m[j][i];
25            for(int k=0; k<tam; k++){
26                m[j][k] = m[j][k]-tmp*m[i][k];
27                id[j][k] = id[j][k]-tmp*id[i][k];
28            }
29        }
30    } return id;
31 }
32 vector<double> matporvec(vector<vector<double>> &m,
33     ↪ vector<double> &b){
34     vector<double> ans(sz(b));
35     for(int i=0; i<sz(b); i++)
36         for(int k=0; k<sz(b); k++)
37             ans[i] += m[i][k]*b[k];
38     return ans;
```

► Sistemas de ecuaciones lineales

Dado un sistema de n ecuaciones con m incógnitas. Devuelve el numero de soluciones del sistema 0, 1 o ∞ y si existe al menos una solución la guarda en el vector `ans`. Complejidad: Tiempo $O((n+m)nm)$ - Memoria $O(nm)$.

```
1 const double EPS = 1e-9;
2 const int INF = 2; // it doesn't actually have to be infinity
3   ↪ or a big number
4 int gauss(vector<vector<double>> a, vector<double> &ans){
5     int n = (int) sz(a);
6     int m = (int) sz(a[0]) - 1;
7     vi where(m, -1);
8     for(int col=0, row=0; col<m && row<n; ++col){
9         int sel = row;
10        for (int i=row; i<n; ++i)
11            if(abs(a[i][col])>abs(a[sel][col]))sel=i;
12        if(abs(a[sel][col]) < EPS) continue;
13        for(int i=col; i<=m; ++i)
14            swap(a[sel][i], a[row][i]);
15        where[col] = row;
16        for(int i=0; i<n; ++i)
17            if(i != row){
18                double c = a[i][col] / a[row][col];
19                for(int j=col; j<=m; ++j)
20                    a[i][j] -= a[row][j] * c;
21            } ++row;
22    } ans.assign(m, 0);
23    for(int i=0; i<m; ++i) if(where[i] != -1)
24        ans[i] = a[where[i]][m] / a[where[i]][i];
25    for(int i=0; i<n; ++i){
26        double sum = 0;
27        for(int j=0; j<m; ++j)
28            sum += ans[j] * a[i][j];
29        if(abs(sum - a[i][m]) > EPS) return 0;
30    }
31    for(int i=0; i<m; ++i) if(where[i] == -1)
32        return INF;
33    return 1;
```

► Sistemas de ecuaciones modulo 2

Dado un sistema de n ecuaciones con m incógnitas modulo 2. Devuelve el numero de soluciones del sistema 0, 1 o ∞ y si existe al menos una solución la guarda en el vector `ans`. Complejidad: Tiempo $O((n+m)nm)$ Memoria $O(nm)$.

```
1 int gauss (vector<bitset<MAXN>> A, int n, int m, bitset<MAXN>
2   ↪ &ans){
3     // where[i] guarda el indice de la ecaucion donde se puede
4     ↪ despejar variable i, -1 si es independiente
5     vi where(m, -1);
6     // Matriz "diagonal"
7     for(int col=0, row=0; col<m && row<n; ++col){
8         // busca pivote
9         for(int i = row; i<n; ++i) if(A[i][col]){
10            swap (A[i], A[row]);
11            break;
12        } if(!A[row][col]) continue;
13        where[col] = row;
14        // poner 0 todos lados
15        for(int i=0; i<n; ++i)
16            if(i != row && A[i][col]) A[i] ^= A[row];
17        ++row;
18    }
19    // sacar respuesta // variables independientes = 1, t
20    for(int i = 0; i < m; ++i)
21        ans[i] = where[i] == -1 ? 1 : ((A[where[i]].count() ^
22            ↪ 1) & 1);
23    // si hay alguna variable libre
```

```
21     for(int i = 0; i < m; ++i) if(where[i] == -1)
22         return INF;
23     return 1;
24 }
```

SPARSE TABLE

Complejidad: Tiempo de precalculo $O(n \log n)$ - Tiempo en responder $O(\log(r-l+1))$ - Tiempo en responder para operaciones idempotentes $O(1)$ - Memoria extra $O(n \log n)$. LOGN es $\lceil \log_2(\text{MAXN}) \rceil$. Indexado en 0.

```
1 struct sparse_table{
2     int n, NEUTRO;
3     vvi ST;
4     vi lg2;
5     int f(int a, int b){return a + b;}
6     sparse_table(int _n, int data[]){
7         n = _n;
8         NEUTRO = 0;
9         lg2.resize(n + 1);
10        lg2[1] = 0;
11        for(int i=2; i<=n; ++i)lg2[i]=lg2[i/2]+1;
12        ST.resize(lg2[n] + 1, vi(n + 1, NEUTRO));
13        for(int i=0; i<n; ++i)ST[0][i] = data[i];
14        for(int k = 1; k <= lg2[n]; ++k){
15            int fin = (1 << k) - 1;
16            for(int i = 0; i + fin < n; ++i)
17                ST[k][i]=f(ST[k-1][i],
18                    ↪ ST[k-1][i+(1<<(k-1))]);
19        }
20    }
21    int query(int l, int r){
22        if(l > r) return NEUTRO;
23        int ans = NEUTRO;
24        for(int k = lg2[n]; 0 <= k; --k){
25            if( r - l + 1 < (1 << k) ) continue;
26            ans = f(ans, ST[k][l]);
27            l += 1 << k;
28        }
29        return ans;
30    }
31    int queryIdem(int l, int r){
32        if(l > r) return NEUTRO;
33        int lg = lg2[r - l + 1];
34        return f(ST[lg][l], ST[lg][r-(1<<lg)+1]);
35    }
36 };
```

FENWICK TREE

Complejidad: Tiempo en responder $O(\log n)$ - Tiempo de actualización $O(\log n)$ - Memoria extra $O(n)$. Indexado en 1.

```
1 struct fenwick_tree{
2     int n;
3     vi BIT;
4     fenwick_tree(int _n){
5         n = _n;
6         BIT.resize(n + 1);
7     }
8     void add(int pos, int x){
9         while(pos <= n){
10            BIT[pos] += x;
11            pos += lsb(pos);
12        }
13    }
14    int sum(int pos){
15        int res = 0;
16        while(pos){
17            res += BIT[pos];
```

```

18     pos -= lsb(pos);
19     } return res;
20 }
21 };

```

SEGMENT TREE

```

1 struct node{
2     int val, lazy;
3     node():val(0), lazy(0){}
4     node(int x, int lz = 0):val(x), lazy(lz){}
5     const node operator+(const node &b) const{
6         return node(val + b.val);
7     }
8 }

```

► Actualizaciones puntuales

Complejidad: Tiempo de precalculo $O(n)$ - Tiempo en responder $O(\log n)$ - Tiempo de actualización $O(\log n)$ - Memoria extra $O(n)$. Indexado en 1.

```

1 struct segment_tree{
2     struct node{...};
3     vector<node> nodes;
4     segment_tree(int n, int data[]){
5         nodes.resize(4 * n + 1);
6         build(1, n, data);
7     }
8     void build(int left, int right, int data[], int pos = 1){
9         if(left == right){
10             nodes[pos] = node(data[left]);
11             return;
12         }
13         int mid = (left + right) / 2;
14         build(left, mid, data, pos * 2);
15         build(mid + 1, right, data, pos * 2 + 1);
16         nodes[pos] = nodes[pos*2] + nodes[pos*2+1];
17     }
18     void update(int x, int idx, int left, int right, int pos = 1){
19         ↪ 1){
20             if(idx < left || right < idx) return;
21             if(left == right){
22                 nodes[pos].val += x;
23                 return;
24             }
25             int mid = (left + right) / 2;
26             update(x, idx, left, mid, pos * 2);
27             update(x, idx, mid+1, right, pos*2+1);
28             nodes[pos] = nodes[pos*2] + nodes[pos*2+1];
29         }
30         node query(int l, int r, int left, int right, int pos = 1){
31             if(r < left || right < l) return node();
32             if(l <= left && right <= r)
33                 return nodes[pos];
34             int mid = (left + right) / 2;
35             return query(l, r, left, mid, pos*2) + query(l, r,
36                 ↪ mid+1, right, pos*2+1);
37         }
38 };

```

► Actualizaciones sobre rangos

Complejidad: Tiempo de precalculo $O(n)$ - Tiempo en responder $O(\log n)$ - Tiempo de actualización $O(\log n)$ - Memoria extra $O(n)$.

```

1 struct segment_tree{
2     struct node{...};
3     vector<node> nodes;
4     segment_tree(int n, int data[]){...}
5     void build(...){...}

```

```

6     void combine_lz(int lz, int pos){nodes[pos].lazy += lz;}
7     void apply_lz(int pos, int tam){
8         nodes[pos].val += nodes[pos].lazy * tam;
9         nodes[pos].lazy = 0;
10    }
11    void push_lz(int pos, int left, int right){
12        int len = abs(right - left + 1);
13        if(1 < len){
14            combine_lz(nodes[pos].lazy, pos*2);
15            combine_lz(nodes[pos].lazy, pos*2+1);
16        } apply_lz(pos, len);
17    }
18    void update(int x, int l, int r, int left, int right, int
19        ↪ pos = 1){
20        push_lz(pos, left, right);
21        if(r < left || right < l) return;
22        if(l <= left && right <= r){
23            combine_lazy(x, pos);
24            push_lazy(pos, left, right);
25            return;
26        }
27        int mid = (left + right) / 2;
28        update(x, l, r, left, mid, pos * 2);
29        update(x, l, r, mid+1, right, pos*2+1);
30        nodes[pos] = nodes[pos*2] + nodes[pos*2+1];
31    }
32    node query(int l, int r, int left, int right, int pos = 1){
33        ↪ push_lz(pos, left, right);
34        ...
35    };

```

► Sparse segment tree

```

1 struct node{
2     ll lazy, maxi, sum;
3     node *left = nullptr, *right = nullptr;
4     node(): lazy(0), maxi(0), sum(0){}
5     void extend(){
6         if(left) return;
7         left = new node;
8         right = new node;
9     }
10    void combine_lz(ll lz){
11        lazy += lz;
12    }
13    void apply_lz(ll len){
14        sum += len * lazy; lazy = 0;
15    }
16    void push_lz(ll L, ll R){
17        int len = R - L + 1;
18        ll mid = L + (R - L) / 2;
19        if(len){
20            extend();
21            left->combine_lz(lazy);
22            right->combine_lz(lazy);
23        }
24        apply_lz(len);
25    }
26    void update(ll x, ll l, ll r, ll L, ll R){
27        push_lz(L, R);
28        if(r < L || R < l) return;
29        if(l <= L && R <= r){
30            combine_lz(x);
31            push_lz(L, R);
32            return;
33        }
34        ll mid = L + (R - L) / 2;
35        extend();
36        left->update(x, l, r, L, mid);
37        right->update(x, l, r, mid + 1, R);
38        sum = left->sum + right->sum;
39    }
40    ll query(ll l, ll r, ll L, ll R){
41        push_lz(L, R);
42        if(r < L || R < l) return 0;
43        if(l <= L && R <= r) return sum;
44        extend();
45        ll mid = L + (R - L) / 2;

```

```

45     return left->query(l, r, L, mid) + right->query(l, r,
46         ↪ mid + 1, R);
47 };

```

SQRT DECOMPOSITION

► Algoritmo de MO

Complejidad: Tiempo en responder $O((n + q)\sqrt{n}F + q \log q)$, donde $O(F)$ es la complejidad de $\text{add}()$ y $\text{remove}()$.

```

1 const int block_size = 300;
2 struct query {
3     int l, r;
4     int block, i;
5     bool operator<(const query &b) const {
6         if(block == b.block) return r < b.r;
7         return block < b.block;
8     }
9 };
10 void add(int idx){/// T0-D0}
11 void remove(int idx){/// T0-D0}
12 int get_answer(){return 0; /// T0-D0}
13 vi solve(vector<query> &queries) {
14     vi answers(sz(queries));
15     sort(queries.begin(), queries.end());
16     int cur_l = 0, cur_r = -1;
17     for(query q : queries){
18         while(cur_l > q.l) add(--cur_l);
19         while(cur_r < q.r) add(++cur_r);
20         while(cur_l < q.l) remove(cur_l--);
21         while(cur_r > q.r) remove(cur_r--);
22         answers[q.i] = get_answer();
23     } return answers;
24 }

```

GRAFOS

```

1 struct edge{
2     int from, to;
3     ll w;
4     const bool operator<(const edge &b) const{
5         return w > b.w;
6     }
7 };
8 struct pos{
9     int from;
10    ll c;
11    const bool operator<(const pos &b) const{
12        return c > b.c;
13    }
14 };

```

► Caminos mínimos

Dijkstra. Complejidad: Tiempo $O(|E| \log |V|)$ - Memoria extra $O(|E|)$.

```

1 ll dijkstra(int a, int b, vector<edge> graph[]){
2     ll dist[MAXN];
3     bool vis[MAXN] = {};
4     fill(dist, dist + MAXN, LLONG_MAX);
5     priority_queue<pos> q;
6     q.push(pos{a, 0}); dist[a] = 0;
7     while(sz(q)){
8         pos cur = q.top(); q.pop();
9         if(vis[cur.from]) continue;
10        vis[cur.from] = true;
11        for(edge &e : graph[cur.from]){
12            if(dist[e.to] <= dist[e.from] + e.w) continue;
13            dist[e.to] = dist[e.from] + e.w;
14            q.push(pos{e.to, dist[e.to]});

```



```

15     }
16     } return dist[b];
17 }

```

Bellman-Ford. Complejidad: $O(|V||E|)$.

```

1 vi bellman_ford(int s, int n, vector<edge> &edges, bool cycles
  ↪ = false){
2     vi d(n, (cycles ? 0 : INT_MAX));
3     d[s] = 0;
4     vi P(n, -1); /// Predecesor
5     for(int i = 0; i < n - 1; ++i)
6         for(edge &e : edges){
7             if(d[e.from] == INT_MAX) continue;
8             if(d[e.to] > d[e.from] + e.w){
9                 d[e.to] = d[e.from] + e.w;
10                P[e.to] = e.from;
11            }
12        }
13    int last_relax = -1;
14    for(edge &e : edges){
15        if(d[e.from] == INT_MAX) continue;
16        if(d[e.to] > d[e.from] + e.w){
17            d[e.to] = d[e.from] + e.w;
18            P[e.to] = e.from;
19            last_relax = e.to;
20        }
21    }
22    if(last_relax == -1) return d;
23    return {}; /// VACIO
24 }

```

Floyd-Warshall. Complejidad: $O(|V|^3)$.

```

1 vvi floyd_warshall(int n){
2     const int INF = INT_MAX; // para evitar overflow después
3     vvi d(n, vi(n, INF));
4     /// aqui inicializa con la lista/matriz de adyacencia
5     /// luego calcula la dp
6     for(int k = 0; k < n; ++k){
7         for(int i = 0; i < n; ++i){
8             for(int j = 0; j < n; ++j){
9                 if(d[i][k] == INF || d[k][j] == INF)
10                    continue;
11                if(d[i][j] > d[i][k] + d[k][j]) d[i][j] =
12                    ↪ d[i][k] + d[k][j];
13            }
14        }
15    }
16    return d;
17 }

```

Johnson's algorithm. Complejidad: $O(|V||E| \log |V|)$.

Sea $p : V \rightarrow \mathbb{R}$ una función potencial del grafo. El algoritmo es como sigue:

1. Hacemos una transformación en el grafo cambiando los pesos w a $w'(u, v) = w(u, v) + p(u) - p(v)$.
2. Calculamos la distancia mínima $d' : V \times V \rightarrow \mathbb{R}$ desde cada nodo a todos los demás con Dijkstra.
3. Finalmente, la distancia mínima de u a v en el grafo original es $d(u, v) = d'(u, v) - p(u) + p(v)$.

La función potencial p puede ser cualquiera. Usando Bellman-Ford se puede calcular el potencial $p(u)$ como el camino más corto que termina (o empieza) en u .

► Árboles

Prim. Complejidad: Tiempo $O(|E| \log |V|)$. `eCost[MAXN]` es el arreglo de costos mínimos de cada nodo para incluirlo en el MST.

```

1 ll prim(vector<edge> graph[]){
2     ll e_cost[MAXN], ans = 0;
3     bool vis[MAXN] = {};
4     fill(e_cost, e_cost + MAXN, LLONG_MAX);
5     priority_queue<edge> q;
6     q.push(edge{1, 1, 0});
7     while(sz(q)){
8         int node = q.top().to;
9         ll w = q.top().w; q.pop();
10        if(vis[node]) continue; vis[node] = true;
11        for(edge &e : graph[node]){
12            if(vis[e.to] || e_cost[e.to] <= e.w) continue;
13            e_cost[e.to] = e.w;
14            q.push(e);
15        } ans += w;
16    } return ans;
17 }

```

Kruskal. Complejidad: Tiempo $O(|E| \log |E|)$.

```

1 ll kruskal(vector<edge> &edges, int n){
2     sort(all(edges));
3     dsu mset(n);
4     ll res = 0;
5     for(edge &e : edges){
6         if(mset.root(e.from) == mset.root(e.to)) continue;
7         mset.join(e.from, e.to);
8         res += e.w;
9     }
10    return res;
11 }

```

Boruvka. Complejidad: Tiempo $O(|E| \log |V|)$. $|V| = n$.

`dsu.join()` devuelve true si la unión se llevó a cabo o false en otro caso.

```

1 ll boruvka(vector<edge> &edges, int n){
2     dsu mset(n);
3     int min_edge[n];
4     ll res = 0;
5     while(mset.cnt_comp > 1){
6         fill(min_edge, min_edge + n, -1);
7         for(int i = 0; i < sz(edges); ++i){
8             int u = mset.root(edges[i].from);
9             int v = mset.root(edges[i].to);
10            if(u == v) continue;
11            if(min_edge[u] == -1 || edges[i].w <
12                ↪ edges[min_edge[u]].w) min_edge[u] = i;
13            if(min_edge[v] == -1 || edges[i].w <
14                ↪ edges[min_edge[v]].w) min_edge[v] = i;
15        }
16        for(int i = 0; i < n; ++i){
17            int idx_e = min_edge[i];
18            if(idx_e == -1) continue;
19            res += mset.join(edges[idx_e].from,
20                ↪ edges[idx_e].to) * edges[idx_e].w;
21        }
22    } return res;
23 }

```

MST dirigido. Complejidad: $O(|E| \log |V|)$. AGREGAR PEQUEÑA DESCRIPCIÓN.

```

1 /// MEJORAR ESTA COSA, SOLO LO COPIE Y
  ↪ PEGUÉ por cuestiones de tiempo

```

```

2 const int N = 3e5 + 9;
3 const ll inf = 1e18;
4 template<typename T> struct PQ {
5     ll sum = 0;
6     priority_queue<T, vector<T>, greater<T>> Q;
7     void push(T x) { x.w -= sum; Q.push(x); }
8     T pop() { auto ans = Q.top(); Q.pop(); ans.w += sum;
9         ↪ return ans; }
10    int size() { return sz(Q); }
11    void add(ll x) { sum += x; }
12    void merge(PQ &x){
13        if (size() < sz(x)){
14            swap(sum, x.sum);
15            Q.swap(x.Q);
16        }
17        while(sz(x)){
18            auto tmp = x.pop();
19            tmp.w -= sum;
20            Q.push(tmp);
21        }
22    };
23    struct edge {
24        int u, v; ll w;
25        bool operator > (const edge &rhs) const { return w >
26            ↪ rhs.w; }
27    };
28    struct DSU {
29        vi par;
30        DSU(int n) : par(n, -1) {}
31        int root(int i) { return par[i] < 0 ? i : par[i] =
32            ↪ root(par[i]); }
33        void set_par(int c, int p) { par[c] = p; }
34    };
35    // returns parents of each vertex
36    // each edge should be distinct
37    // it assumes that a solution exists (all vertices are
38    ↪ reachable from root)
39    // 0 indexed
40    // Takes ~300ms for n = 2e5
41    vi DMST(int n, int root, const vector<edge> &edges) {
42        vi u(2 * n - 1, -1), par(2 * n - 1, -1);
43        edge par_edge[2 * n - 1];
44        vi child[2 * n - 1];
45        PQ<edge> Q[2 * n - 1];
46        for(auto e : edges) Q[e.v].push(e);
47        for(int i = 0; i < n; i++){
48            Q[(i+1) % n].push({i, (i+1) % n, inf});
49        }
50        int super = n;
51        DSU dsu(2 * n - 1);
52        int head = 0;
53        while(sz(Q[head])) {
54            auto x = Q[head].pop();
55            int nxt_root = dsu.root(x.u);
56            if (nxt_root == head) continue;
57            u[head] = nxt_root;
58            par_edge[head] = x;
59            if (u[nxt_root] == -1) head = nxt_root;
60            else {
61                int j = nxt_root;
62                do {
63                    Q[j].add(-par_edge[j].w);
64                    //Q[super].merge(move(Q[j]));
65                    Q[super].merge(Q[j]);
66                    assert(u[j] != -1);
67                    child[super].pb(j);
68                    j = dsu.root(u[j]);
69                } while (j != nxt_root);
70                for(auto u : child[super]) par[u] = super;
71                ↪ dsu.set_par(u, super);
72                head = super++;
73            }
74        }
75        vi res(2 * n - 1, -1);
76        queue<int> q; q.push(root);
77        while (sz(q)) {
78            int u = q.front();
79            q.pop();

```

```

75 while (par[u] != -1) {
76     for (auto v : child[par[u]]) {
77         if (v != u) {
78             res[par_edge[v].v] = par_edge[v].u;
79             q.push(par_edge[v].v);
80             par[v] = -1;
81         }
82     }
83     u = par[u];
84 }
85 }
86 res[root] = root; res.resize(n);
87 return res;
88 }
89 int main() {
90     ios_base::sync_with_stdio(0); cin.tie(0);
91     int n, m, root; cin >> n >> m >> root;
92     vector<edge> edges(m);
93     for(auto &e : edges) cin >> e.u >> e.v >> e.w;
94     auto res = DMST(n, root, edges);
95     unordered_map<int, int> W[n];
96     for (auto u : edges) W[u.v][u.u] = u.w;
97     ll ans = 0;
98     for(int i = 0; i < n; i++) if(i != root)
99         ans += W[i][res[i]];
100     cout << ans << '\n';
101     for(auto x : res) cout << x << ' ';
102     cout << '\n';
103 }

```

Block-Cut Tree Complejidad: $O(n)$?

```

1 /// MEJORAR ESTA COSA, SOLO LO COPIE Y
2   ↳ PEGUÉ por cuestiones de tiempo
3 vvi comps; // nodes in each component
4 vvi bcc; // nodes to components that it belong
5 vi low, num;
6 int id;
7 int sz;
8 void dfs_biconnected(vvi &g, int u, int pre){
9     low[u] = num[u] = ++id;
10    st.pb(u);
11    for(auto v: g[u]) {
12        if(!num[v]) {
13            dfs_biconnected(g, v, u);
14            low[u] = min(low[u], low[v]);
15            if(low[v] >= num[u]) {
16                int x;
17                comps.pb(vi());
18                do {
19                    x = st.back();
20                    st.pop_back();
21                    bcc[x].pb(sz);
22                    comps.back().pb(x);
23                } while(x ^ v);
24                bcc[u].pb(sz);
25                comps.back().pb(u);
26                sz++;
27            }
28        } else if(v != pre) low[u] = min(low[u], num[v]);
29    }
30 }
31 vi utoubt; // its component or its AP index
32 vb uisart; // u is AP?
33 vvi bt; // block cut tree
34 void generateBlockCutTree(vvi &g){
35     int n = sz(g);
36     sz = id = 0;
37     bcc.resize(0); low.resize(0); num.resize(0);
38     bcc.resize(n); low.resize(n); num.resize(n);
39     dfs_biconnected(g, 0, 0);
40     bt.resize(sz);
41     utoubt.resize(0); utoubt.resize(n);
42     uisart.resize(0); uisart.resize(n);
43     for(int u = 0; u < n; u++){
44         if(sz(bcc[u]) > 1) { //Articulation
45             utoubt[u] = sz++;

```

```

46         uisart[u] = true;
47         bt.pb(vi());
48         for(auto v: bcc[u]){
49             bt[utoubt[u]].pb(v);
50             bt[v].pb(utoubt[u]);
51         }
52     } else //Not articulation point
53         utoubt[u] = bcc[u][0];
54 }
55 }

```

LCA. Complejidad: Tiempo de preproceso $O(|V| \log |V|)$.
Tiempo de LCA y n -ésimo ancestro $O(\log |V|)$. 1-index.

```

1 void precalc(int node, int p = 0, int d = 1){
2     depth[node] = d;
3     P[0][node] = p;
4     for(int k = 1; k <= LOGN; ++k)
5         P[k][node] = P[k-1][P[k-1][node]];
6     for(int child : tree[node]) if(p != child)
7         precalc(child, node, d + 1);
8 }
9 int LCA(int a, int b){
10    if(depth[b] < depth[a]) swap(a, b);
11    int dif = depth[b] - depth[a];
12    for(int k = LOGN; 0 <= k; --k)
13        if(is_on(dif, k)) b = P[k][b];
14    if(a == b) return a;
15    for(int k = LOGN; 0 <= k; --k){
16        if(P[k][a] != P[k][b]){
17            a = P[k][a];
18            b = P[k][b];
19        }
20    }
21    return P[0][a];
22 }
23 int nth_ancestor(int u, int n){
24     for(int k = LOGN; 0 <= k; --k)
25         if(is_on(n, k)) u = P[k][u];
26     return u;
27 }

```

Sack. Complejidad: Tiempo $O(|V| \log |V|)$. 1-index.

```

1 void precalc(int node, int p = 0){
2     subtree_size[node] = 1;
3     depth[node] = depth[p] + 1;
4     for(int v : tree[node]){
5         if(v == p) continue;
6         precalc(v, node);
7         subtree_size[node] += subtree_size[v];
8     }
9 }
10 void add(int node, int x, int p = 0){
11     // add node here
12     // add subtree
13     for(int v: tree[node])
14         if(v != p && !big[v])
15             add(v, x, node);
16 }
17 void dfs(int node, bool keep, int p = 0){
18     int maxi = -1, big_child = -1;
19     for(int v : tree[node]) //Search for big_child
20         if(v != p && subtree_size[v] > maxi)
21             maxi = subtree_size[v], big_child = v;
22     for(int v : tree[node])
23         if(v != p && v != big_child)
24             dfs(v, false, node); // run a dfs on small childs
25     // and clear them
26     if(big_child != -1)
27         dfs(big_child, true, node), big[big_child] = 1; //
28     // big_child marked as big and not cleared
29     add(node, 1, p);
30     // answer queries here
31     if(big_child != -1) big[big_child] = 0;
32     if(!keep) add(node, -1, p);
33 }

```

► Máximo flujo

Algunos problemas de flujos

- *Maximum Weight Closure.* Sea N_1 una clausura de G y $N_2 = V \setminus N_1$, tenemos que $w(N_1) = \sum_{i \in N_1^+} w_i - \sum_{i \in N_1^-} |w_i|$ y $Cap.Corte = \sum_{i \in N_2^+} w_i + \sum_{i \in N_1^-} |w_i|$. Entonces

$$Cap.Corte + w(N_1) = \sum_{i \in N_1^+} w_i + \sum_{i \in N_2^+} w_i.$$

- *Mínima cobertura de vértices.* En grafos generales es NP-Completo. En grafos bipartitos el máximo emparejamiento es igual al número de vértices en la mínima cobertura. Para el problema con pesos en los nodos, unimos s a todos los nodos en L con capacidad igual al peso de cada nodo, unimos los nodos de R a t de la misma manera y unimos los nodos de L a R con capacidad infinita. El máximo flujo es el peso mínimo de la mínima cobertura.

- *Máximo conjunto independiente.* Cualquier conjunto independiente es el complemento de alguna cobertura de vértices.

- *Mínimo cubrimiento de caminos independientes.* En grafos generales es NP-hard. En DAG's duplicamos los nodos en un lado IN y un lado OUT. Conectamos s al lado OUT y el lado IN a t . Las aristas del DAG las agregamos del lado OUT al lado IN. Sea M el máximo emparejamiento de la red anterior, entonces el mínimo cubrimiento es $|V| - M$.

- *Mínimo cubrimiento de caminos NO necesariamente independientes.* En grafos generales es NP-hard. En DAG's transformamos el DAG a su clausura transitiva y aplicamos el problema anterior.

- *Teorema de Menger.* Sea G un grafo finito (sin pesos) con vértices s, t distintos. El corte mínimo (por aristas) entre s y t es igual a la cantidad máxima de caminos independientes por aristas de s a t . Si s y t no son adyacentes, el corte mínimo por vértices entre s y t es igual a la cantidad máxima de caminos independientes por vértices de s a t .

- *Teorema de Mirsky.* En todo POSET, el tamaño de la cadena de mayor tamaño es igual al número de anticadenas necesarias para cubrir todos los elementos del conjunto.
- *Teorema de Dilworth.* En todo POSET, el tamaño de la anticadena de mayor tamaño es igual al número de cadenas necesarias para cubrir todos los elementos del conjunto.
- *Teorema de Hall.* Un grafo bipartito con subconjuntos L y R tiene un emparejamiento de tamaño $|L|$ si y sólo si para todo subconjunto W de L , se cumple que $|W| \leq |N_G(W)|$, donde $N_G(W)$ es el conjunto de vértices vecinos de alguno de los vértices en W .

Edmonds-Karp Complejidad: Ford-Fulkerson $O(|E| \cdot F)$, Edmonds-Karp $O(|V||E|^2)$.

```

1 struct edge {
2     int from, to;
3     ll w, c, f;
4     // weight, capacity, flow
5 };
6 class ford_fulkerson {
7 public:
8     ford_fulkerson (vector<vector<edge>> &graph) :
9         graph(graph){}
10    ll get_max_flow(int s, int t){
11        init();
12        ll f = 0;
13        while(find_and_update(s, t, f)){
14            return f;
15        }
16    vi get_st_cut(const int &s){
17        bool vis[sz(graph)] = {};
18        vi S;
19        queue<int> q;
20        q.push(s);
21        S.pb(s);
22        vis[s] = true;
23        while(sz(q)){
24            int u = q.front(); q.pop();
25            for(int eI : edge_indexes[u]){
26                if(edges[eI].c > edges[eI].f &&
27                    !vis[edges[eI].to]){
28                    q.push(edges[eI].to);
29                    vis[edges[eI].to] = true;
30                    S.pb(edges[eI].to);
31                }
32            }
33        }
34        return S;
35    }
36    vector<vector<edge>> get_residual_graph(){
37        vector<vector<edge>> residual(sz(graph));
38        for(int i=0; i<sz(edges); i+=2){
39            const edge& e = edges[i];
40            if(e.c > 0){
41                residual[e.from].pb({e.from, e.to, e.w, e.c -
42                    e.f, e.f});
43                residual[e.to].pb({e.to, e.from, -e.w, e.f,
44                    -e.f});
45            }
46        }
47        return residual;
48    }
49 }

```

```

44 private:
45     vector<vector<edge>> graph;
46     vector<edge> edges;
47     vvi edge_indexes;
48     void init(){
49         edges.clear();
50         edge_indexes.clear();
51         edge_indexes.resize(sz(graph));
52         for(int u = 0; u < sz(graph); u++){
53             for(edge &e : graph[u]){
54                 edges.pb({u, e.to, e.w, e.c, 0});
55                 edges.pb({e.to, u, -e.w, 0, 0});
56                 edge_indexes[u].pb(sz(edges)-2);
57                 edge_indexes[e.to].pb(sz(edges)-1);
58             }
59         }
60     }
61     bool find_and_update(int s, int t, ll &flow){
62         queue<int> q;
63         // Desde donde llego y con que arista
64         vpii from(sz(graph), {-1, -1});
65         q.push(s);
66         from[s] = {s, -1};
67         bool found = 0;
68         while(sz(q) && !found){
69             int u = q.front(); q.pop();
70             for(int eI : edge_indexes[u]){
71                 if(edges[eI].c > edges[eI].f &&
72                     from[edges[eI].to].fi == -1){
73                     from[edges[eI].to] = {u, eI};
74                     q.push(edges[eI].to);
75                     if(edges[eI].to == t) found=1;
76                 }
77             }
78         }
79         if(!found) return false;
80         ll u_flow = LLONG_MAX;
81         int cur = t;
82         while(cur != s){
83             u_flow = min(u_flow, edges[from[cur].se].c -
84                 edges[from[cur].se].f);
85             cur = from[cur].fi;
86         }
87         cur = t;
88         while(cur != s){
89             edges[from[cur].se].f += u_flow;
90             edges[from[cur].se].f -= u_flow;
91             cur = from[cur].fi;
92         }
93         flow += u_flow;
94         return 1;
95     }
96 }

```

Dinic. Complejidad: $O(|V|^2|E|)$.

```

1 const int MAXV = 32767; // 2^15 - 1
2 template<class T = ll> struct dinic{
3     dinic(short V){this->V = V;}
4     const static bool SCALING = 1;
5     bool sorted = 0;
6     short s, t, V;
7     int lim = 1; // Para escalado
8     const T INF = numeric_limits<T>::max();
9     short level[MAXV]; // distancia desde s
10    short ptr[MAXV]; // arista que va explorando
11    struct edge{
12        short to, rev;
13        T cap, flow, mcap;
14        bool operator<(const edge &b) const {return mcap >
15            b.mcap;}
16    };
17    vector<edge> adj[MAXV];
18    vi adj_cur[MAXV]; // aristas del grafo de nivel
19    void add_edge(short u, short v, T cap, bool is_directed =
20        1){
21        if(u == v) return;
22        T add = (is_directed ? 0 : cap);
23    }

```

```

21     adj[u].pb({v, sz(adj[v]), cap, 0, cap+add});
22     adj[v].pb({u, sz(adj[u])-1, add, 0, cap+add});
23 }
24 void mysort(){
25     if(sorted) return;
26     sorted = 1;
27     for(int i = 0; i < V; ++i){
28         sort(all(adj[i]));
29         for(int j=0; j<sz(adj[i]); ++j)
30             adj[adj[i][j].to][adj[i][j].rev].rev = j;
31     }
32 }
33 bool bfs(){
34     for(int i = 0; i < V; ++i){
35         adj_cur[i].clear();
36         adj_cur[i].reserve(sz(adj[i]));
37     }
38     queue<short> q;
39     q.push(s);
40     fill(level, level + V, -1);
41     level[s] = 0;
42     while(sz(q)){
43         short u = q.front(); q.pop();
44         if(u == t) return 1;
45         for(int i=0; i < sz(adj[u]); ++i){
46             edge &e = adj[u][i];
47             if(e.mcap < lim) break;
48             if(level[e.to] == -1 && e.cap - e.flow >= lim){
49                 level[e.to] = level[u] + 1;
50                 adj_cur[u].pb(i);
51                 q.push(e.to);
52             } else if(level[e.to] == level[u]+1 && e.cap -
53                 e.flow >= lim){
54                 adj_cur[u].pb(i);
55             }
56         }
57     }
58     return 0;
59 }
60 T dfs(short u, T flow, vector<short> &S, bool save = 0){
61     if(save) S.pb(u);
62     if(u == t) return flow;
63     for(ptr[u] < sz(adj_cur[u]); ++ptr[u]){
64         edge &e = adj[u][adj_cur[u][ptr[u]]];
65         if(T pushed = dfs(e.to, min(flow, e.cap - e.flow),
66             S, save)){
67             e.flow += pushed;
68             adj[e.to][e.rev].flow -= pushed;
69             if(e.cap - e.flow < lim) ptr[u]++;
70             return pushed;
71         }
72     }
73     return 0;
74 }
75 ll get_max_flow(short source, short sink){
76     s = source; t = sink;
77     mysort();
78     vector<short> S;
79     ll flow = 0;
80     lim = SCALING ? (1<<30) : 1;
81     for(0 < lim; lim >= 1){
82         while(bfs()){
83             memset(ptr, 0, sizeof(ptr));
84             while(T pushed = dfs(s, INF, S)){
85                 flow += pushed;
86             }
87         }
88     }
89     return flow;
90 }
91 vector<short> get_st_cut(){
92     vector<short> S;
93     memset(ptr, 0, sizeof(ptr));
94     dfs(s, INF, S, true);
95     return S;
96 }

```

MCMF. Complejidad: $O(\min\{|E|^2|V| \log |V|, F|E| \log |V|\})$

```

1 const int MAXV = (1 << 15) - 1;
2 template<class T = ll> struct mcmf{

```

```

3  mcmf(short V){this->V = V;}
4  short s, t, V;
5  const T INF = numeric_limits<T>::max();
6  vector<T> p;
7  struct edge{
8      short to, rev;
9      T cap, flow, mcap, w;
10     bool operator<(const edge &b)const{return mcap >
11         ↳ b.mcap;}
12 };
13 vector<edge> adj[MAXV];
14 void add_edge(short u, short v, T cap, T w, bool
15     ↳ is_directed = 1){
16     if(u == v) return;
17     T add = (is_directed ? 0 : cap);
18     adj[u].pb({v, sz(adj[v]), cap, 0, cap + add, w});
19     adj[v].pb({u, sz(adj[u]) - 1, add, 0, cap + add, -w});
20 }
21 struct pos{
22     short from;
23     T c;
24     const bool operator<(const pos &b)const{return c > b.c;}
25 };
26 vector<T> bellman_ford(){
27     vector<T> d(V);
28     for(int i = 0; i < V - 1; ++i){
29         for(int u = 0; u < V; ++u){
30             for(edge &e : adj[u]){
31                 if(d[e.to] > d[u] + e.w && e.cap > 0){
32                     d[e.to] = d[u] + e.w;
33                 }
34             }
35         }
36     } return d;
37 }
38 pair<T, T> dijkstra(const T MAX_FLOW){
39     vector<T> d(V, INF);
40     vi P(V, -1), P_e(V, -1);
41     priority_queue<pos> q;
42     q.push({s, 0});
43     d[s] = 0;
44     while(sz(q)){
45         pos act = q.top(); q.pop();
46         if(act.c != d[act.from]) continue;
47         for(int j=0; j<sz(adj[act.from]); ++j){
48             edge &e = adj[act.from][j];
49             if(e.cap - e.flow <= 0) continue;
50             T _w = e.w + p[act.from] - p[e.to];
51             if(d[e.to] <= d[act.from] + _w)
52                 continue;
53             d[e.to] = d[act.from] + _w;
54             q.push(pos{e.to, d[e.to]});
55             P[e.to] = act.from;
56             P_e[e.to] = j;
57         }
58     }
59     for(int i = 0; i < V; ++i) if(d[i] < INF)
60         d[i] += -p[s] + p[i];
61     for(int i = 0; i < V; ++i) if(d[i] < INF)
62         p[i] = d[i];
63     if(P[t] == -1) return {0, 0};
64     T flow = MAX_FLOW;
65     int cur_node = t;
66     while(cur_node != s){
67         flow = min(flow, adj[ P[cur_node] ][ P_e[cur_node]
68             ↳ ].cap - adj[ P[cur_node] ][ P_e[cur_node]
69             ↳ ].flow);
70         cur_node = P[cur_node];
71     }
72     T new_cost = 0;
73     cur_node = t;
74     while(cur_node != s){
75         adj[ P[cur_node] ][ P_e[cur_node] ].flow += flow;
76         new_cost += adj[ P[cur_node] ][ P_e[cur_node] ].w
77             ↳ * flow;
78         adj[cur_node][adj[ P[cur_node] ][ P_e[cur_node]
79             ↳ ].rev ].flow -= flow;
80         cur_node = P[cur_node];

```

```

75     }
76     return {flow, new_cost};
77 }
78 pair<T, T> get_max_flow(short source, short sink, const T
79     ↳ MAX_FLOW = 1e8){
80     s = source;
81     t = sink;
82     p = bellman_ford();
83     T flow = 0, cost = 0;
84     while(flow < MAX_FLOW){
85         pair<T, T> pushed = dijkstra(MAX_FLOW - flow);
86         if(!pushed.fi) break;
87         flow += pushed.fi;
88         cost += pushed.se;
89     }
90     return {flow, cost};
91 }

```

► SCC

Kosajaru Complejidad: Tiempo $O(n)$.

```

1  void dfs(int u, vi graph[], bool vis[], vi &topo_ord){
2      if(vis[u]) return;
3      vis[u] = true;
4      for(int v : graph[u])dfs(v, graph, vis, topo_ord);
5      topo_ord.pb(u);
6  }
7  void assign_scc(int u, vi inv_graph[], bool vis[], vi &scc,
8      ↳ const int id){
9      if(vis[u]) return;
10     vis[u] = true;
11     scc[u] = id;
12     for(int v : inv_graph[u])
13         assign_scc(v, inv_graph, vis, scc, id);
14 }
15 pair<int, vi> kosajaru(int n, vi graph[], vi inv_graph[]){
16     bool vis[n] = {};
17     vi topo_ord;
18     for(int i=0; i<n; ++i) dfs(i, graph, vis, topo_ord);
19     reverse(all(topo_ord));
20     memset(vis, 0, sizeof(vis));
21     vi scc(n);
22     int id = 0;
23     for(int u : topo_ord) if(!vis[u])
24         assign_scc(u, inv_graph, vis, scc, id++);
25     return {id, scc};
26 }
27 pair<vvi, vvi> build_scc_graph(int n, vi graph[], int max_scc,
28     ↳ const vi &scc){
29     vvi scc_graph(max_scc), inv_scc_graph(max_scc);
30     for(int u = 0; u < n; ++u)
31         for(int v : graph[u])
32             if(scc[u] != scc[v])
33                 scc_graph[scc[u]].pb(scc[v]);
34     for(int u = 0; u < n_scc; ++u){
35         sort(all(scc_graph[u]));
36         auto it = unique(all(scc_graph[u]));
37         scc_graph[u].resize(it - scc_graph[u].begin());
38     }
39     for(int v : scc_graph[u])
40         inv_scc_graph[v].pb(u);
41     return {scc_graph, inv_scc_graph};

```

► 2-Sat

Complejidad: Tiempo en responder $O(n)$.

```

1  struct two_sat{
2      int n;
3      vvi graph, inv_graph;
4      vi scc, ans;
5      vector<bool> vis;
6      two_sat(){}
7      two_sat(int _n){

```

```

8          n = _n;
9          graph.resize(2 * n);
10         inv_graph.resize(2 * n);
11         scc.resize(2 * n);
12         vis.resize(2 * n);
13         ans.resize(n);
14     }
15     void add_edge(int u, int v){
16         graph[u].pb(v);
17         inv_graph[v].pb(u);
18     }
19     /// al menos una es verdadera
20     void add_or(int p, bool val_p, int q, bool val_q){
21         add_edge(p+(val_p? n:0), q+(val_q? 0:n));
22         add_edge(q+(val_q? n:0), p+(val_p? 0:n));
23     }
24     /// exactamente una es verdadera
25     void add_xor(int p, bool val_p, int q, bool val_q){
26         add_or(p, val_p, q, val_q);
27         add_or(p, !val_p, q, !val_q);
28     }
29     /// p y q tienen el mismo valor
30     void add_and(int p, bool val_p, int q, bool val_q){
31         add_xor(p, !val_p, q, val_q);
32     }
33     /// Kosajaru
34     void dfs(int node, vi &topo_ord){...}
35     void assign_scc(int node, const int id){...}
36     /// construye respuesta
37     bool build_ans(){
38         fill(all(vis), false);
39         vi topo_ord;
40         for(int i=0; i<2*n; ++i)dfs(i, topo_ord);
41         fill(all(vis), false);
42         reverse(all(topo_ord));
43         int id = 0;
44         for(int u : topo_ord) if(!vis[u])
45             assign_scc(u, id++);
46         for(int i = 0; i < n; ++i){
47             if(scc[i] == scc[i + n]) return 0;
48             ans[i] = (scc[i] < scc[i + n] ? 0:1);
49         }
50         return 1;
51     }

```

TREAP

Complejidad: Tiempo $O(\log(n))$ - Memoria $O(n)$. Para treap implícito (arreglo dinamico) cambiar en insert/erase a split_by_pos().

```

1  struct treap{
2      typedef struct _node{
3          ll x;
4          int freq, cnt;
5          ll p;
6          _node *l, *r;
7          _node(ll _x): x(_x), p(((ll)rand()) << 32)^rand(),
8              cnt(1), freq(1), l(nullptr), r(nullptr){}
9          _node(){delete l; delete r;}
10         void recalc(){
11             cnt = freq;
12             cnt += (l ? (l->cnt) : 0);
13             cnt += (r ? (r->cnt) : 0);
14         }
15     }* node;
16     node root;
17     void merge(node l, node r){
18         if(!l || !r) return l ? l : r;
19         if(l->p < r->p){
20             r->l = merge(l, r->l);
21             r->recalc();
22             return r;
23         } else {
24             l->r = merge(l->r, r);

```



```

25     l->recalc();
26     return l;
27 }
28 }
29 void split_by_value(node n, ll d, node &l, node &r){
30     l = r = nullptr;
31     if(!n) return;
32     if(n->x < d){
33         split_by_value(n->r, d, n->r, r);
34         l = n;
35     } else {
36         split_by_value(n->l, d, l, n->l);
37         r = n;
38     }
39     n->recalc();
40 }
41 void split_by_pos(node n, int pos, node &l, node &r, int
42 ↪ l_nodes = 0){
43     l = r = NULL;
44     if(!n) return;
45     int cur_pos = (n->l ? (l_nodes + n->l->cnt) : l_nodes;
46     if(cur_pos < pos){
47         split_by_pos(n->r, pos, n->r, r, cur_pos + 1);
48         l = n;
49     } else {
50         split_by_pos(n->l, pos, l, n->l, l_nodes);
51         r = n;
52     }
53     n->recalc();
54 }
55 treap(): root(NULL){}
56 void insert_value(ll x){
57     node l, m, r;
58     split_by_value(root, x, l, m);
59     split_by_value(m, x + 1, m, r);
60     if(m){
61         m->freq++;
62         m->cnt++;
63     } else m = new _node(x);
64     root = merge(merge(l, m), r);
65 }
66 void erase_value(ll x){
67     node l, m, r;
68     split_by_value(root, x, l, m);
69     split_by_value(m, x + 1, m, r);
70     if(!m || m->freq == 1){
71         delete m;
72         m = nullptr;
73     } else {
74         m->freq--;
75         m->cnt--;
76     }
77     root = merge(merge(l, m), r);
78 };

```

STRINGS

► KMP

Complejidad: Tiempo $O(|s|)$ - Memoria extra $O(|s|)$.

```

1 vi prefix_function(const string &s){
2     int n = sz(s);
3     vi pi(n);
4     for(int i = 1; i < n; ++i){
5         int j = pi[i - 1];
6         while(j && s[i] != s[j]) j = pi[j - 1];
7         pi[i] = j + (s[i] == s[j]);
8     } return pi;
9 }

```

Autómata de KMP. Complejidad: Tiempo $O(|s|k)$ - Memoria extra $O(|s|k)$, donde k es el tamaño del alfabeto.

```

1 void compute_automaton(const string &s, vvi& aut){

```

```

2     s += '#';
3     int n = sz(s);
4     vi pi = prefix_function(s);
5     aut.assign(n, vi(26));
6     for(int i = 0; i < n; ++i){
7         for(int c = 0; c < 26; ++c){
8             if(i && 'a' + c != s[i])
9                 aut[i][c] = aut[pi[i - 1]][c];
10            else aut[i][c] = i + ('a' + c == s[i]);
11        }
12    }
13 }

```

► Suffix Automata

Complejidad: Tiempo $O(|s|)$, memoria $O(|S|)$. Construye un autómata sobre los sufijos de s , donde cada nodo terminal es un sufijo de s . Esta construcción se hace online. Donde α es el tamaño del alfabeto y $offset$ el primer caracter. Si se hace uso de los nodos terminales llamar a `markTerminalNodes()` primero (este método no es online).

```

1 constexpr short alpha = 26;
2 constexpr char offset = 'a';
3 struct state{
4     //max(state) = len, min(state) = sa[link].len
5     int len, link;
6     bool is_terminal;
7     array<int, alpha> next;
8     state(){
9         len = 0; link = -1; is_terminal = false;
10        next.fill(0);
11    }
12 };
13 struct SuffixAutomaton{
14     int n;
15     vector<state> sa;
16     int sz = 1, last = 0;
17     SuffixAutomaton(int n) : n(n), sa(2*n + 1){}
18     void add(char ch){
19         short ch = ch - offset;
20         int curr = sz++;
21         sa[curr].len = sa[last].len + 1;
22         int p = last;
23         while(p != -1 && !sa[p].next[ch]){
24             sa[p].next[ch] = curr;
25             p = sa[p].link;
26         }
27         if(p == -1){
28             sa[curr].link = 0;
29         } else {
30             int q = sa[p].next[ch];
31             if(sa[p].len + 1 == sa[q].len){
32                 sa[curr].link = q;
33             } else {
34                 int clone = sz++;
35                 sa[clone].len = sa[p].len + 1;
36                 sa[clone].next = sa[q].next;
37                 sa[clone].link = sa[q].link;
38                 while(p != -1 && sa[p].next[ch] == q){
39                     sa[p].next[ch] = clone;
40                     p = sa[p].link;
41                 }
42                 sa[q].link = sa[curr].link = clone;
43             }
44         }
45         last = curr;
46     }
47     return;
48 }
49 void markTerminalNodes(){
50     int curr = last;
51     while(curr){
52         sa[curr].is_terminal = true;
53         curr = sa[curr].link;

```

```

53     }
54     return;
55 }
56 };

```

► Suffix array

Construcción. Complejidad: Tiempo $O(|s| \log(|s|))$ - Memoria $O(|s|)$. Calcula la permutación que corresponde a los sufijos ordenados lexicográficamente. $sa[i]$ es el índice en el cual empieza el i -ésimo sufijo ordenado.

```

1 int SA[MAXN], mrank[MAXN];
2 int tmpSA[MAXN], tmp_mrank[MAXN];
3 void counting_sort(int k, int n){
4     int freqs[MAXN] = {};
5     for(int i = 0; i < n; ++i){
6         if(i + k < n) freqs[mrank[i + k]]++;
7         else freqs[0]++;
8     }
9     int m = max(100, n);
10    for(int i = 0, sfs = 0; i < m; ++i){
11        int f = freqs[i];
12        freqs[i] = sfs;
13        sfs += f;
14    }
15    for(int i = 0; i < n; ++i){
16        if(SA[i] + k < n) tmpSA[freqs[mrank[SA[i] + k]]++] =
17            SA[i];
18        else tmpSA[freqs[0]++] = SA[i];
19    }
20    for(int i = 0; i < n; ++i) SA[i] = tmpSA[i];
21 }
22 void buildSA(string &str){
23     int n = sz(str);
24     for(int i = 0; i < n; ++i){
25         mrank[i] = str[i] - '#';
26         SA[i] = i;
27     }
28     for(int k = 1; k < n; k <= 1){
29         counting_sort(k, n);
30         counting_sort(0, n);
31         int r = 0;
32         tmp_mrank[SA[0]] = 0;
33         for(int i = 1; i < n; ++i){
34             if(mrank[SA[i]] != mrank[SA[i - 1]] || mrank[
35                 ↪ SA[i] + k] != mrank[SA[i - 1] + k])
36                 tmp_mrank[SA[i]] = ++r;
37             else tmp_mrank[SA[i]] = r;
38         }
39         for(int i = 0; i < n; ++i) mrank[i] = tmp_mrank[i];
40     }
41 }
42 inline bool suff_compare1(int idx, const string &pattern) {
43     return (s.substr(idx).compare(0, sz(pattern), pattern) < 0);
44 }
45 inline bool suff_compare2(const string &pattern, int idx) {
46     return (s.substr(idx).compare(0, sz(pattern), pattern) > 0);
47 }
48 pair<int, int> match(const string &pattern) {
49     int *low = lower_bound(SA, SA + sz(s), pattern,
50         ↪ suff_compare1);
51     int *up = upper_bound(SA, SA + sz(s), pattern,
52         ↪ suff_compare2);
53     return make_pair((int)(low - SA), (int)(up - SA));

```

Prefijo común más largo. Complejidad: Tiempo $O(|s|)$ - Memoria $O(|s|)$. Calcula la longitud del prefijo común más largo entre dos sufijos consecutivos (lexicográficamente) de s . $lcp[i]$ guarda la respuesta para el i -ésimo sufijo y el $(i - 1)$ -ésimo sufijo.


```

1 int lcp[MAXN];
2 void build_lcp(string &str){
3     int n = sz(str);
4     int phi[n];
5     phi[SA[0]] = -1;
6     for(int i = 1; i < n; ++i) phi[ SA[i] ] = SA[i - 1];
7     int plcp[n];
8     int k = 0;
9     for(int i = 0; i < n; ++i){
10        if(phi[i] == -1){
11            plcp[i] = 0;
12            continue;
13        }
14        while(i + k < n && phi[i] + k < n && str[i + k] ==
15            str[phi[i] + k]) k++;
16        plcp[i] = k;
17        k = max(k - 1, 0);
18    }
19    for(int i = 0; i < n; ++i) lcp[i] = plcp[SA[i]];

```

► Aho-Corasick

Construcción en $O(mk)$, donde m es el tamaño total de los strings y k el tamaño del alfabeto.

```

1 struct aho_corasick{
2     const static int K = 26;
3     const char index = 'a';
4     struct vertex{
5         int next[K];
6         bool terminal = false;
7         int p = -1;
8         char p_edge;
9         int link = -1;
10        int terminal_link = -1;
11        int go[K];
12        vertex(int p = -1, char c = '$') : p(p), p_edge(c){
13            memset(next, -1, K*sizeof(int));
14            memset(go, -1, K*sizeof(int));
15        }
16    };
17    vector<vertex> aho;
18    aho_corasick(){ aho.resize(1); }
19    void add_string(const string &s){
20        int u = 0;
21        for(char c : s){
22            int e = c - index;
23            if(aho[u].next[e] == -1){
24                aho[u].next[e] = sz(aho);
25                aho.emplace_back(u, c);
26            }
27            u = aho[u].next[e];
28        } aho[u].terminal = 1;
29    }
30    int get_link(int u){
31        if(aho[u].link == -1){
32            if(!u || !aho[u].p) aho[u].link = 0;
33            else aho[u].link = go(get_link(aho[u].p),
34                aho[u].p_edge);
35        } return aho[u].link;
36    }
37    int go(int u, char c){
38        int e = c - index;
39        if(aho[u].go[e] == -1){
40            if(aho[u].next[e] != -1)
41                aho[u].go[e] = aho[u].next[e];
42            else aho[u].go[e] = !u ? 0 : go(get_link(u), c);
43        } return aho[u].go[e];
44    }
45    int get_terminal_link(int u){
46        if(aho[u].terminal_link == -1){
47            if(!u || !aho[u].p)
48                aho[u].terminal_link = 0;
49            else aho[u].terminal_link =
50                get_terminal_link(get_link(u));
51        } return aho[u].terminal_link;

```

```

50    }
51 };

► Suffix tree
1 /// MEJORAR ESTA COSA, SOLO LO COPIE Y
2   ↳ PEGUÉ porcuacionesdetiempo
3 const int inf = 1e9;
4 const int maxn = 1e6;
5 int s[maxn];
6 map<int, int> to[maxn];
7 //Root is the vertex 0
8 //f_pos[i] is the initial index with the letter of the edge
9   ↳ that goes from the parent of i to i
10 //len[i] is the number of letters in the edge that enters in i
11 //slink[i] is the suffix link
12 int len[maxn], f_pos[maxn], slink[maxn];
13 int node, pos;
14 int sz = 1, n = 0;
15 void make_node(int _pos, int _len){
16     f_pos[sz] = _pos;
17     len[sz] = _len;
18     return sz++;
19 }
20 void go_edge(){
21     while(pos > len[to[node][s[n - pos]]]){
22         node = to[node][s[n - pos]];
23         pos -= len[node];
24     }
25 }
26 void add_letter(int c){
27     s[n++] = c;
28     pos++;
29     int last = 0;
30     while(pos > 0){
31         go_edge();
32         int edge = s[n - pos];
33         int &v = to[node][edge];
34         int t = s[f_pos[v] + pos - 1];
35         if(v == 0){
36             v = make_node(n - pos, inf);
37             //v = make_node(n - pos, 1);
38             slink[last] = node;
39             last = 0;
40         } else if(t == c) {
41             slink[last] = node;
42             return;
43         } else {
44             int u = make_node(f_pos[v], pos - 1);
45             to[u][c] = make_node(n - 1, inf);
46             to[u][t] = v;
47             f_pos[v] += pos - 1;
48             len[v] -= pos - 1;
49             v = u;
50             slink[last] = u;
51             last = u;
52         }
53     }
54     if(node == 0) pos--;
55     else node = slink[node];
56 }
57 void correct(int s_size){
58     len[0] = 0;
59     for(int i = 1; i < sz; i++){
60         if(f_pos[i] + len[i] - 1 >= s_size){
61             len[i] = (s_size - f_pos[i]);
62         }
63     }
64 }
65 void print_suffix_tree(int from){
66     cout << "Edge entering in " << from << " has size " <<
67     len[from];
68     cout << " and starts in " << f_pos[from] << endl;
69     cout << "Node " << from << " goes to: ";
70     for(auto u : to[from]){
71         cout << u.se << " with " << (char)u.fi << " ";
72     }
73     cout << endl;

```

```

71     for(auto u : to[from]){
72         print_suffix_tree(u.se);
73     }
74 }
75 void build(string &s){
76     for(int i = 0; i < sz; i++){
77         to[i].clear();
78     }
79     sz = 1;
80     node = pos = n = 0;
81     len[0] = inf;
82     for(int i = 0; i < sz(s); i++){
83         add_letter(s[i]);
84         correct(sz(s));
85     }
86 void cutGeneralized(vi &finishPoints){
87     for(int i = 0; i < sz; i++){
88         int init = f_pos[i];
89         int end = f_pos[i] + len[i] - 1;
90         int idx = lower_bound(all(finishPoints), init) -
91             finishPoints.begin();
92         if((idx != sz(finishPoints)) && (finishPoints[idx] <=
93             end)){//Must be cut
94             len[i] = (finishPoints[idx] - f_pos[i] + 1);
95             to[i].clear();
96         }
97     }
98 void build_generalized(vector<string> &ss){
99     for(int i = 0; i < sz; i++){
100         to[i].clear();
101     }
102     sz = 1;
103     node = pos = n = 0;
104     len[0] = inf;
105     int sep = 256;
106     vi finishPoints;
107     int next = 0;
108     for(int i = 0; i < sz(ss); i++){
109         for(int j = 0; j < sz(ss[i]); j++){
110             add_letter(ss[i][j]);
111         }
112         next += sz(ss[i]);
113         finishPoints.pb(next);
114         add_letter(sep++);
115         next++;
116     }
117     correct(next);
118     cutGeneralized(finishPoints);

```

GEOMETRÍA

► Convex hull

Complejidad: $O(n \log n)$.

```

1 /// MEJORAR ESTA COSA, SOLO LO COPIE Y
2   ↳ PEGUÉ porcuacionesdetiempo
3 struct pt {
4     double x, y;
5 };
6 int orientation(pt a, pt b, pt c) {
7     double v = a.x*(b.y-c.y)+b.x*(c.y-a.y) + c.x*(a.y-b.y);
8     if(v < 0) return -1; // clockwise
9     if(v > 0) return +1; // counter-clockwise
10    return 0;
11 }
12 bool cw(pt a, pt b, pt c, bool include_collinear){
13     int o = orientation(a, b, c);
14     return o < 0 || (include_collinear && !o);
15 }
16 bool collinear(pt a, pt b, pt c){
17     return orientation(a, b, c) == 0;
18 }
19 void convex_hull(vector<pt> &a, bool include_collinear = 0){
20     pt p0 = *min_element(all(a), [](pt a, pt b){
21         return make_pair(a.y, a.x) < make_pair(b.y, b.x);

```

```

20 });
21 sort(all(a), [&p0](const pt& a, const pt& b){
22     int o = orientation(p0, a, b);
23     if(o == 0)
24         return (p0.x-a.x)*(p0.x-a.x) + (p0.y-a.y)*(p0.y-a.y)
25             < (p0.x-b.x)*(p0.x-b.x) + (p0.y-b.y)*(p0.y-b.y);
26     return o < 0;
27 });
28 if(include_collinear){
29     int i = sz(a)-1;
30     while(i >= 0 && collinear(p0, a[i], a.back())) i--;
31     reverse(a.begin()+i+1, a.end());
32 }
33 vector<pt> st;
34 for(int i = 0; i < sz(a); i++){
35     while(sz(st)>1 && !cw(st[sz(st)-2], st.back(), a[i],
36         ↪ include_collinear))
37         st.pop_back();
38     st.pb(a[i]);
39 }
40 a = st;

```

UTILIDADES

► Subset sum optimization.

Complejidad: $O(N\sqrt{N})$. Si queremos obtener calcular el conjunto de sumas posibles en subconjuntos de un arreglo A , en el cual se cumple que $\sum a_i = N$ y todo $a_i \geq 0$, podemos obtener exactamente el mismo conjunto con un arreglo comprimido de tamaño $\sqrt{N}$. El $3k - trick$ explota que las sumas generadas por el arreglo $[a, a, a]$ son exactamente las mismas que en $[2a, a]$. Podemos juntar elementos y repetirlo hasta que a solo haya dos copias de cada elemento distinto en el nuevo arreglo, acotando el numero de elementos a $O(\sqrt{N})$. El proceso de compresión toma $O(N \log_2(N))$.

```

1 vi compress(vector<ll>& a){
2     sort(a.rbegin(), a.rend());
3     int n = sz(a);
4     a.pb(0);
5     vi weights;
6     int pi = 0;
7     for(int i = 1; i <= n; i++){
8         if(a[i] != a[i-1]){
9             ll cnt = i - pi;
10            ll x = a[i-1];
11            ll j = 1;
12            while(j < cnt){
13                weights.pb(x * j);
14                cnt -= j;
15                j *= 2;
16            }
17            weights.pb(x * cnt);
18            pi = i;
19        }
20    } return weights;
21 }

```

► Bitsets de tamaño (casi) dinámico

El código ejecutado dentro de func(sz) usará un bitset con tamaño de la potencia de 2 más cercano a sz.

```
1 template<int len = 1> void func(int sz){
```

```

2     if(sz > len){
3         func<std::min(len * 2, MAXLEN)>(sz);
4         return;
5     }
6     bitset<len> bs;
7     //usa aquí tu bitset dinámico
8 }

```

► Bitwise ops

```

1 #define is_on(S, j) (S & (1ll << (j)))
2 #define set_bit(S, j) (S |= (1ll << (j)))
3 #define clear_bit(S, j) (S &= ~(1ll << (j)))
4 #define toggle_bit(S, j) (S ^= (1ll << (j)))
5 #define lsb(S) ((S) & ~(S))
6 #define clear_lsb(S) (S &= (S - 1))
7 #define set_all(S, n) (S = (1ll << (n)) - 1ll)
8 #define clear_trailing_ones(S) (S &= (S + 1))
9 #define set_last_bit_off(S) (S |= (S + 1))
10 #define is_power_of_two(S) (!(S) & ((S) - 1))
11 #define nearest_power_of_two(S) ((int)pow(2,
12     ↪ (int)((log((double)(S)) / log(2)) + 0.5)) )
13 #define is_divisible_by_power_of_two(n, k) !((n) & ((1ll <<
14     ↪ (k)) - 1))
15 #define modulo(S, N) ((S) & ((N) - 1)) // S%N, N=2^k

```

► Bitwise - builtin

```

1 // one plus the index of the least significant 1-bit of x, or
2 // if x is zero, returns zero.
3 int __builtin_ffs(int x)
4 // number of leading 0-bits in x, starting at the most
5 // significant bit position. If x is 0 is undefined
6 int __builtin_clz(unsigned int x)
7 // number of trailing 0-bits in x, starting at the least
8 // significant bit position. If x is 0 undefined.
9 int __builtin_ctz(unsigned int x)
10 // the parity of x, i.e. the number of 1-bits in x modulo 2.
11 int __builtin_parity(unsigned int x)

```

► Iterar

Dada una máscara m, iterar sobre todos sus subconjuntos desde el más grande hasta el más pequeño (ojo el código no itera sobre $x = m$). La complejidad de iterar sobre todas las submáscaras de todos los números de 1 a 2^n es $O(3^n)$.

```

1 void submasks(int mask){
2     for( int x = mask; x; ){
3         --x &= mask;
4     }
5 }

```

► Gossers' Hack

Sirve para generar todos las máscaras de n bits, que tengan exactamente k bits a 1 (y que sean menores o iguales que 2^n).

Complejidad $O\left(\binom{n}{k}\right)$

```

1 void GossersHack(int n, int k) {
2     int mask = (1 << k) - 1;
3     int limit = (1 << n);
4     while(mask < limit){
5         int c = mask & - mask;
6         int r = mask + c;
7         mask = (((r ^ mask) >> 2) / c) | r;
8     }
9 }

```

► Subset Sum con bitset

Dado una una arreglo de tamaño n ver si es posible encontrar es la suma objetivo S . Nota: w es el tamaño del bloque de bits procesados en paralelo (normalmente 32 o 64). Complejidad: Temporal $O\left(n \times \frac{S}{w}\right)$ - Memoria $O(S)$

```

1 bool subsetSumBitset( vi& arr, int S) {
2     bitset<10001> dp; // tam >= S+1
3     dp[0] = 1;
4     for(int x : arr) dp |= (dp << x);
5     return dp[S];
6 }

```

MÁXIMO DE FUNCIONES

► Li-Chao Tree

Dado un conjunto A con M valores a evaluar, y N funciones (tales que cada una de ellas se intersecta con el resto a lo más una vez), te devuelve $\max_{i \in [N]} (f_i(a))$ para cualquier $a \in A$. Complejidad: Responder y agregar $O(\log M)$.

```

1 struct Function {
2     ll m;
3     ll b;
4     ll eval(ll x){
5         if (m == LLONG_MIN) return LLONG_MIN;
6         return m*x+b;
7     }
8     Function(){ m = LLONG_MIN; }
9     Function(ll m_, ll b_): m(m_), b(b_){ }
10 };
11 struct LiChaoTree{
12     vll values;
13     ll maxV;
14     Function *functions;
15     LiChaoTree(vll &values){
16         values = values;
17         sort(all(values));
18         functions = new Function[sz(values)*4];
19         maxV = sz(values);
20     }
21     //Range from l to r - 1
22     ll get(ll x){
23         return get(x, 1, 0, maxV);
24     }
25     ll get(ll x, int v, int l, int r){
26         int m = l + (r - l) / 2;
27         ll mv = values[m];
28         if(r - l == 1){
29             return functions[v].eval(x);
30         } else if(x < mv){
31             return max(functions[v].eval(x), get(x, 2 * v, l,
32                 ↪ m));
33         } else {
34             return max(functions[v].eval(x), get(x, 2 * v + 1,
35                 ↪ m, r));
36         }
37     }
38     void addFunction(Function f){
39         addFunction(f, 1, 0, maxV);
40     }
41     void addFunction(Function f, int v, int l, int r){
42         int m = l + (r - l) / 2;
43         ll mv = values[m];

```

```
42     ll lv = values[l];
43     bool lef = f.eval(lv) > functions[v].eval(lv);
44     bool mid = f.eval(mv) > functions[v].eval(mv);
45     if(mid)//Si el actual pierde en el medio
46         swap(functions[v], f);
47     if(r - l == 1) return;
48     else if(lef != mid)//El cruce esta en izq
49         addFunction(f, 2 * v, l, m);
50     else addFunction(f, 2 * v + 1, m, r);
51 }
52 ~LiChaoTree(){ delete[] functions; }
53 };
```